

將 Vertex AI 代理與 Google Workspace 整合

來源：<https://codelabs.developers.google.com/vertexai-gws-agents?hl=zh-tw>

步驟數：7

在本程式碼研究室中，您將建構與 Google Workspace 緊密整合的 Vertex AI 服務專員，方法是透過資料儲存庫、Google 管理的 Model Context Protocol (MCP) 伺服器、Workspace API 和 Google Workspace 外掛程式。這些範例是使用 Gemini 模型、Vertex AI Search 和 Agent Engine、Agent Development Kit (ADK) 和 Google Cloud 建構而成。

目錄

1. [1. 事前準備](#)
2. [2. 設定](#)
3. [3. Vertex AI Search 應用程式](#)
4. [4. 自訂代理](#)
5. [5. 以 Google Workspace 外掛程式形式提供的 Agent](#)
6. [6. 清除所用資源](#)
7. [7. 恭喜](#)

1. 事前準備

附註：本程式碼實驗室專為 Vertex AI 設計，如要進一步瞭解如何建構 Google Workspace AI 解決方案，請參閱「[將 Gemini Enterprise 代理與 Google Workspace 整合](#)」和「[Build with AI for Google Workspace](#)」。



什麼是 Vertex AI？

Vertex AI 是 Google Cloud 的統合式開發平台，可供建構、部署及擴充企業級 AI 代理程式和應用程式。這個平台提供進階工具，可供開發人員和資料科學家設計自訂代理工作流程，並與全球規模的基礎架構深度整合。

- **存取 Model Garden**：從超過 150 種基礎模型中選擇，包括完整的 Gemini 系列、第三方模型和專用開放原始碼模型，找出最適合特定代理程式工作的模型。
- **建構複雜的自動化調度管理**：Vertex AI 提供框架，可設計自動代理，運用推論功能規劃及執行多步驟任務，並呼叫外部 API。
- **企業級基準**：將代理連結至即時業務資料，包括高效能 RAG (檢索增強生成) 技術，以消除幻覺並確保事實準確度。
- **DevOps**：透過強大的 SDK、API 和評估工具，將代理程式開發作業無縫整合至現有的 CI/CD 管道，大規模評估代理程式效能和安全性。
- **工業級安全防護**：Vertex AI 會確保用於訓練或建立基準的客戶資料保持私密、經過加密，並符合全球資料落地要求。
- **最佳化基礎架構**：在 Google 世界級的 TPU 和 GPU 叢集上輕鬆擴充代理程式工作負載，即使是全球最嚴苛的應用程式，也能確保低延遲效能。



什麼是 Google Workspace？

Google Workspace 是一套雲端式效率提升與協作解決方案，適用於個人、學校和企業：

- **通訊**：專業電子郵件服務 (Gmail)、視訊會議 (Meet) 和團隊訊息 (Chat)。
- **內容創作**：撰寫文件 (Google 文件)、建立試算表 (Google 試算表) 和設計簡報 (Google 簡報) 的工具。
- **整理**：共用日曆 (日曆) 和數位筆記 (Keep)。
- **儲存空間**：集中式雲端空間，可安全地儲存及共用檔案 (雲端硬碟)。
- **管理**：管理控制項，可管理使用者和安全性設定 (Workspace 管理控制台)。

自訂整合的類型？

Google Workspace 和 Vertex AI 形成強大的回饋循環，Workspace 提供即時資料和協作情境，Vertex AI 則提供自動化智慧工作流程所需的模型、代理功能推論和自動化調度管理。

- **智慧連線：**透過 Google 管理的資料儲存空間、API 和 MCP 伺服器 (Google 管理和自訂)，代理程式可以安全無縫地存取 Workspace 資料，並代表使用者採取行動。
- **自訂代理：**團隊可使用無程式碼設計工具或專業程式碼架構，根據管理員控管的 Workspace 資料和動作，建構專用代理。
- **原生整合：**無論是透過專屬 UI 元件或背景程序，Workspace 外掛程式都能彌合 AI 系統與 Chat 和 Gmail 等應用程式之間的差距。代理人可直接在使用者所在位置提供即時協助，並根據情境提供支援。

結合 Google Workspace 強大的生產力生態系統與 Vertex AI 先進的代理功能，機構就能透過自訂的資料導向 AI 代理，直接在團隊每天使用的工具中自動執行複雜工作流程，進而轉型營運。

必要條件

如要在自己的環境中完成所有步驟，您需要：

- 具備 [Google Cloud](#) 和 [Python](#) 的基本知識。
- 您是擁有者且已啟用計費功能的 Google Cloud 專案。如要確認現有專案是否已啟用計費功能，請參閱「[確認專案的帳單狀態](#)」。如要建立專案及設定帳單，請參閱「[建立 Google Cloud 專案](#)」。如要變更專案擁有權，請參閱「[管理專案成員或變更專案擁有權](#)」。
- 可存取 [Google Chat](#) 且已啟用智慧功能的 Business 或 Enterprise [Google Workspace](#) 帳戶。
- 已安裝 Google Cloud CLI，並為 Google Cloud 雲端專案[初始化](#)。
- 已安裝 Python 3.11 以上版本，請參閱 [Python 官方網站](#)的說明。

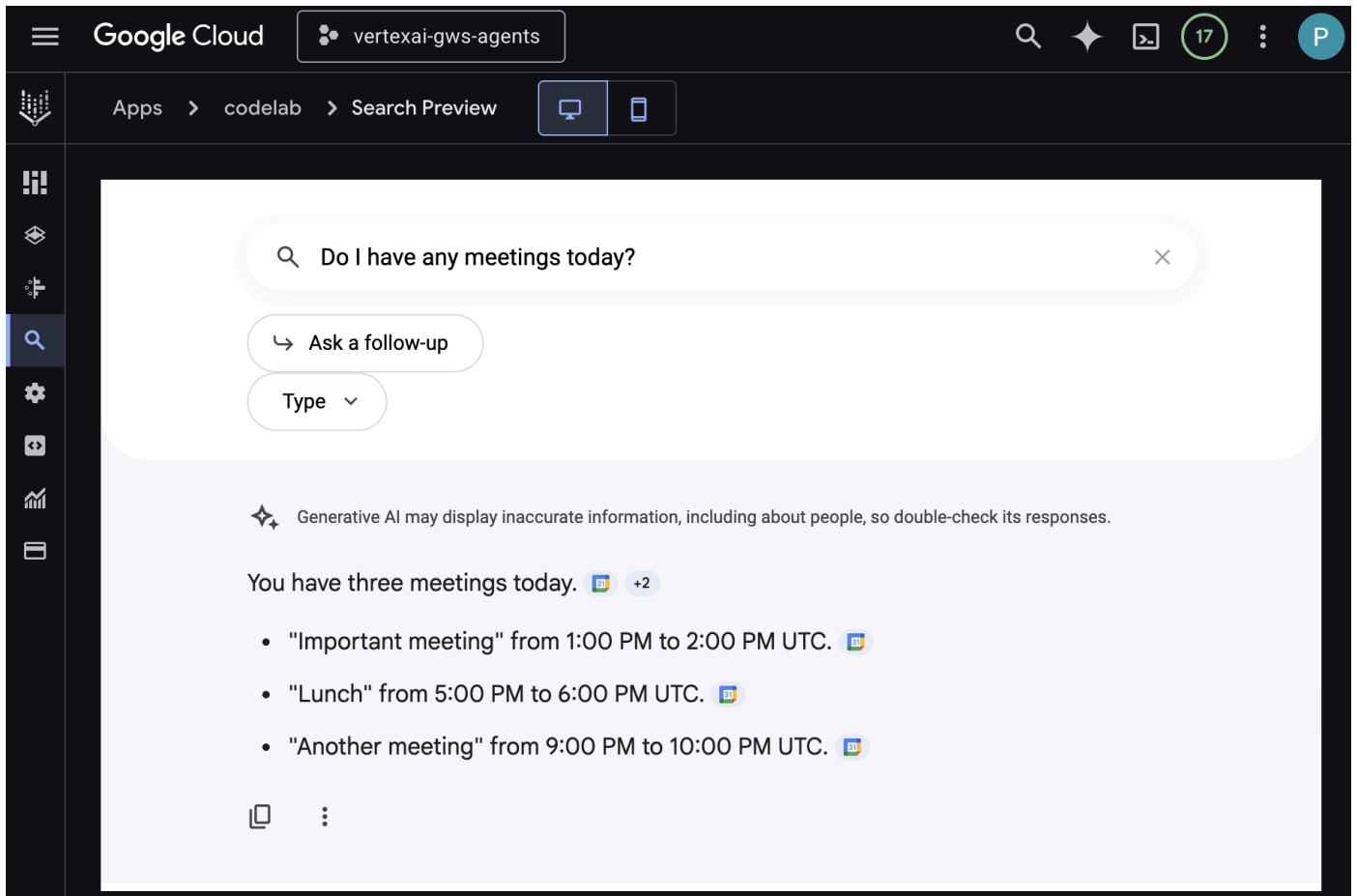
建構目標

在本程式碼研究室中，我們將建構三種解決方案，並將 Vertex AI 代理與 Google Workspace 緊密整合。他們將展示可用於與資料、動作和 UI 互動的架構模式。

Vertex AI Search 應用程式

使用者可以透過自然語言，使用這個代理程式搜尋資料及執行 Workspace 動作。這項功能需要下列元素：

- **模型：**Gemini。
- **資料和動作：**Google Workspace (日曆、Gmail、雲端硬碟) 的 Vertex AI 資料儲存庫。
- **代理主機：**Vertex AI Search。
- **使用者介面：**Vertex AI Search 網頁小工具。



自訂代理程式

使用者可以透過這個代理程式，使用自訂工具和規則，以自然語言搜尋資料及執行 Workspace 動作。這項功能需要下列元素：

- 模型：Gemini。
- 資料和動作：Google Workspace (日曆、Gmail、雲端硬碟) 的 Vertex AI 資料儲存庫、Google 管理的 Vertex AI Search Model Context Protocol (MCP) 伺服器、傳送 Google Chat 訊息的自訂工具函式 (透過 Google Chat API)。
- 代理建構工具：Agent Development Kit (ADK)。
- 代理主機：Vertex AI Agent Engine。
- 使用者介面：ADK Web。

#1

Please find my meetings for today, I need their titles and links



#2



⚡ search

#3



✓ search

Here are your meetings for today:

#4



- **Lunch** Link: <https://www.google.com/calendar/event?eid=N3V2czVpdTN1MTB2cjlxMHQzcDBoYWNmbjAgbWVAcGllcnJpY2t2b3VsZXQuY29t>
- **Important meeting** Link: <https://www.google.com/calendar/event?eid=MGtrNzF2N3Z0ZHRqMjY1cjJOMDUyMzZnMGYgbWVAcGllcnJpY2t2b3VsZXQuY29t>
- **Another meeting** Link: <https://www.google.com/calendar/event?eid=NXRjcGo1aGJ2c25wM2JwMjJhdGwyZXJlZHYgbWVAcGllcnJpY2t2b3VsZXQuY29t>

#5

Please send a Chat message to someone@example.com with the following text: Hello!



#6



⚡ send_direct_message

#7



✓ send_direct_message

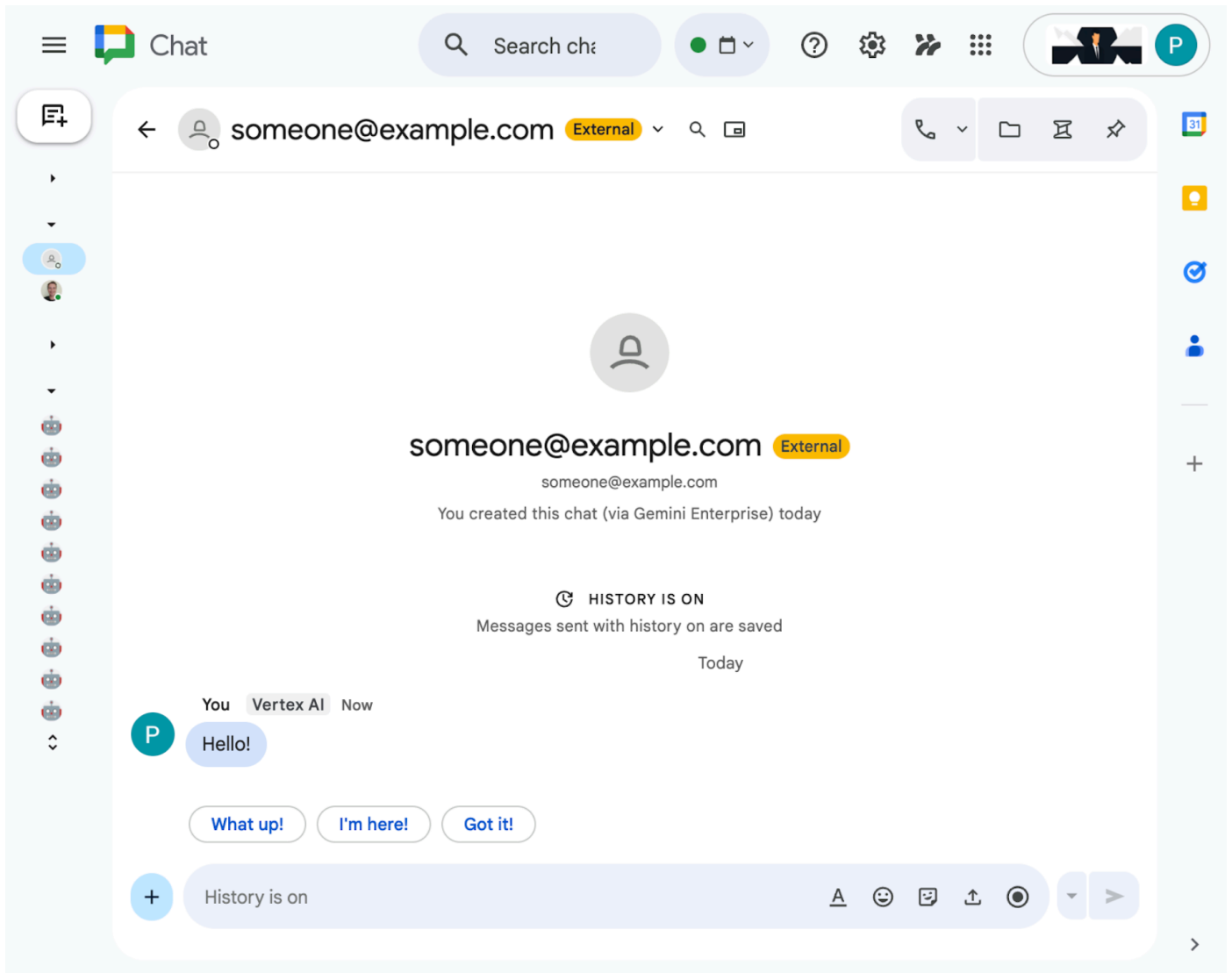
#8



Your message has been sent to someone@example.com.

Type a Message...

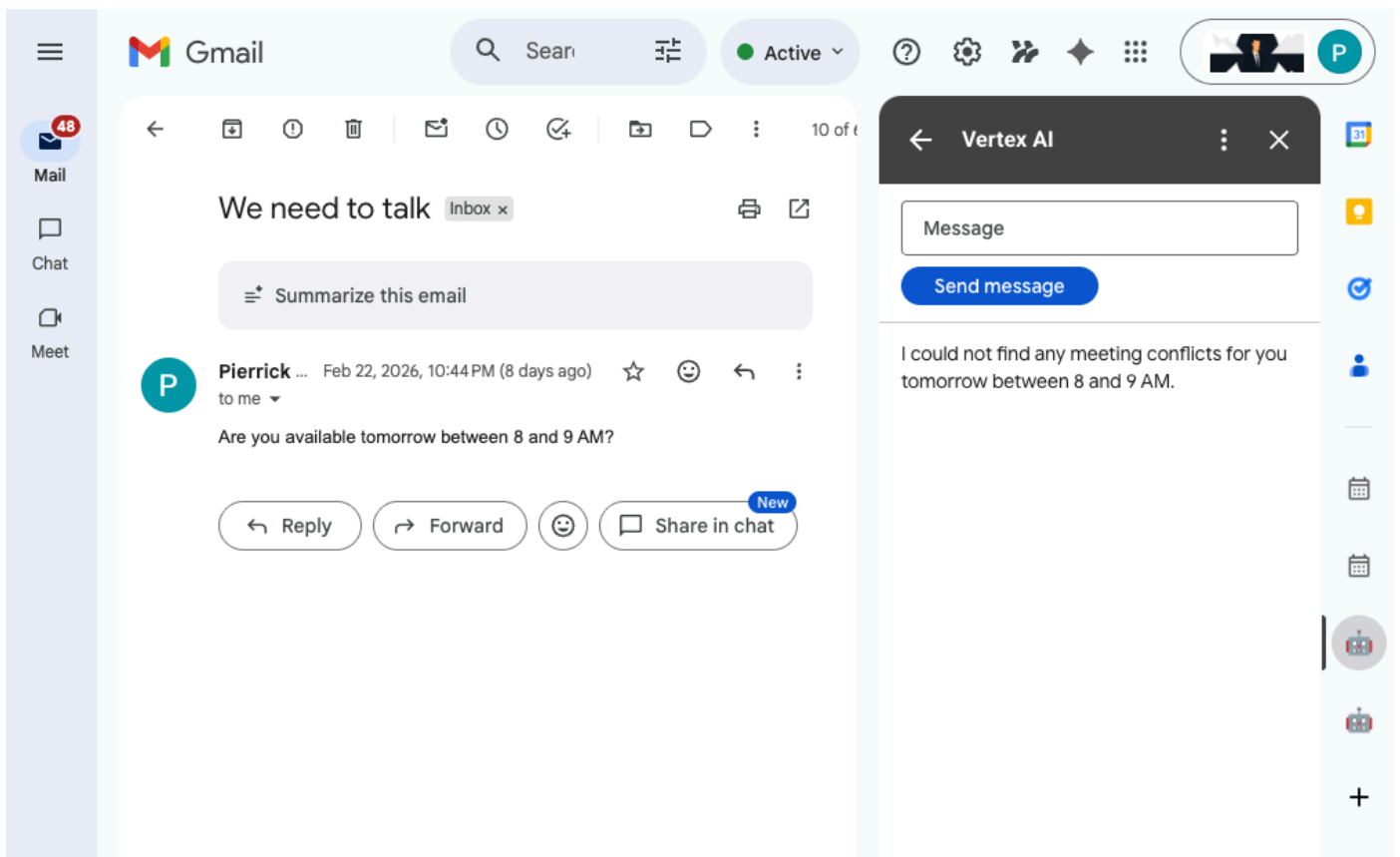
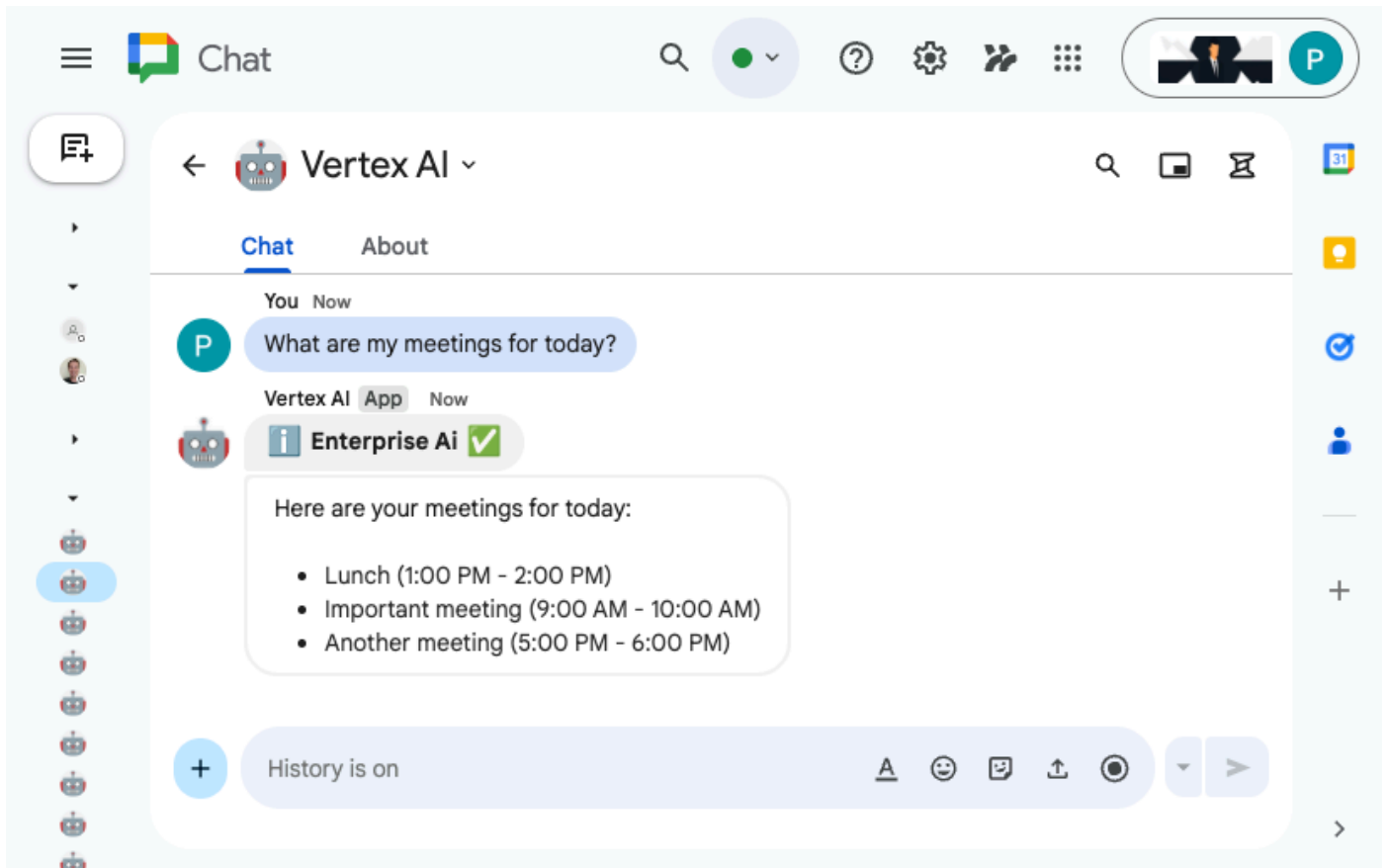




以 Google Workspace 外掛程式形式提供的 Agent

使用者可以在 Workspace 應用程式 UI 中，以自然語言搜尋 Workspace 資料。這項功能需要下列元素：

- 模型：Gemini。
- 資料和動作：Google Workspace (日曆、Gmail、雲端硬碟) 的 Vertex AI 資料儲存庫、Google 管理的 Vertex AI Search Model Context Protocol (MCP) 伺服器、傳送 Google Chat 訊息的自訂工具函式 (透過 Google Chat API)。
- 代理建構工具：Agent Development Kit (ADK)。
- 代理主機：Vertex AI Agent Engine。
- 使用者介面：適用於 Chat 和 Gmail 的 Google Workspace 外掛程式 (可輕鬆擴充至 Google 日曆、雲端硬碟、文件、試算表和簡報)。
- **Google Workspace 外掛程式**：Apps Script、Vertex AI Agent Engine API、情境 (選取的 Gmail 郵件)。



學習目標

- Vertex AI Search 與 Google Workspace 之間的整合點，可啟用資料和動作。
- 在 Vertex AI 中建構自訂代理的選項。
- 使用者存取代理的方式，例如 Vertex AI Search 網頁小工具和 Google Workspace 應用程式。

輕鬆存取這個程式碼實驗室



步驟 2 · 約 10 分鐘

2. 設定

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

建構解決方案前，我們必須先初始化專案的 Vertex AI 應用程式設定、啟用必要 API，以及建立 Vertex AI Workspace 資料儲存庫。

查看概念

Vertex AI 應用程式

[Vertex AI 應用程式](#)是 Google Cloud 上的端對端管理型解決方案，可整合機器學習模型 (例如生成式 AI 代理或搜尋引擎)、企業資料和專用工具，執行語意搜尋、內容生成或自動化客戶互動等複雜工作。

Vertex AI 資料儲存庫

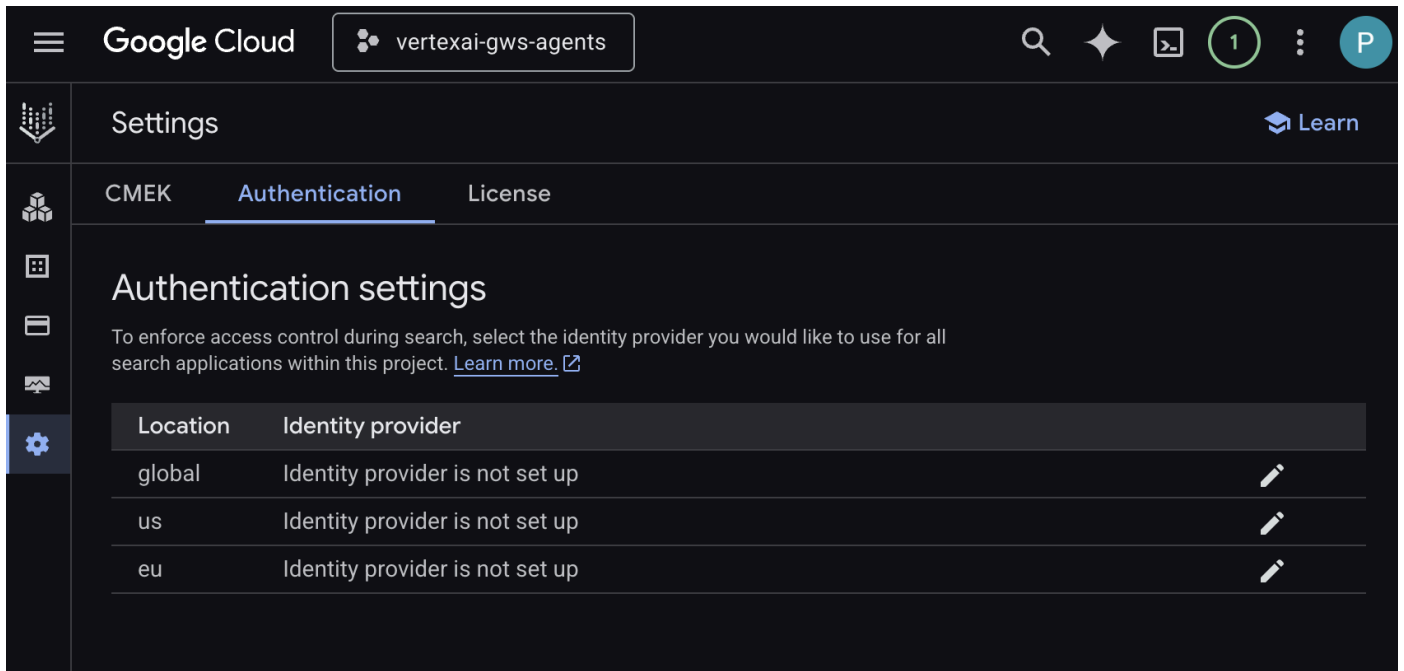
[Vertex AI 資料儲存庫](#)是包含從第一方資料來源 (例如 Google Workspace) 或第三方應用程式 (例如 Jira 或 Shopify) 擷取資料的實體。含有第三方應用程式資料的資料儲存庫也稱為資料連接器。

啟動 Vertex AI 應用程式設定

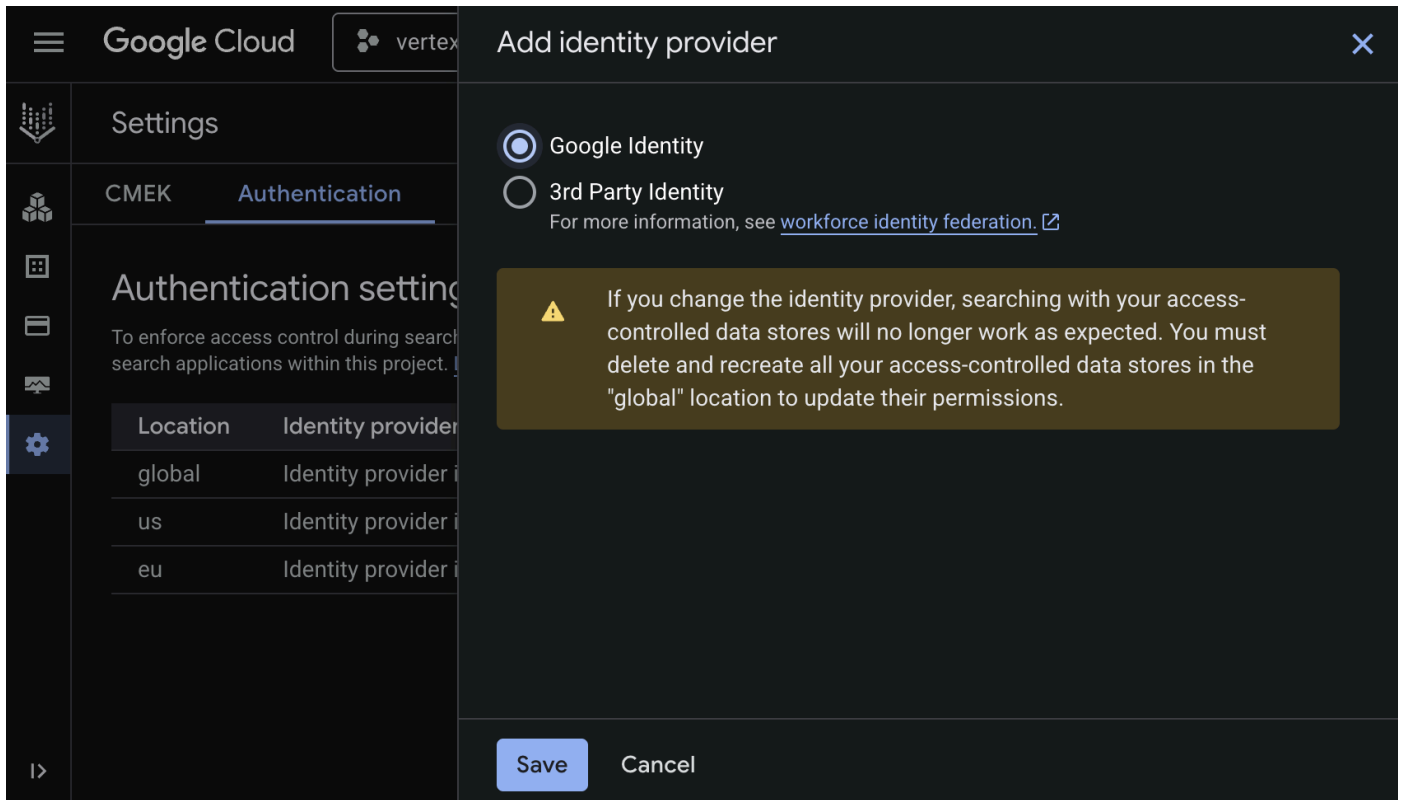
初始化 Vertex AI 應用程式設定，啟用代理程式建立功能。

在新分頁中開啟 [Google Cloud 控制台](#)，然後按照下列步驟操作：

1. 選取專案。
2. 在 Google Cloud 搜尋欄位中，前往「AI Applications」
3. 詳閱並同意條款後，按一下「繼續並啟用 API」。
4. 前往「Settings」。
5. 在「Authentication」分頁中，編輯「global」。



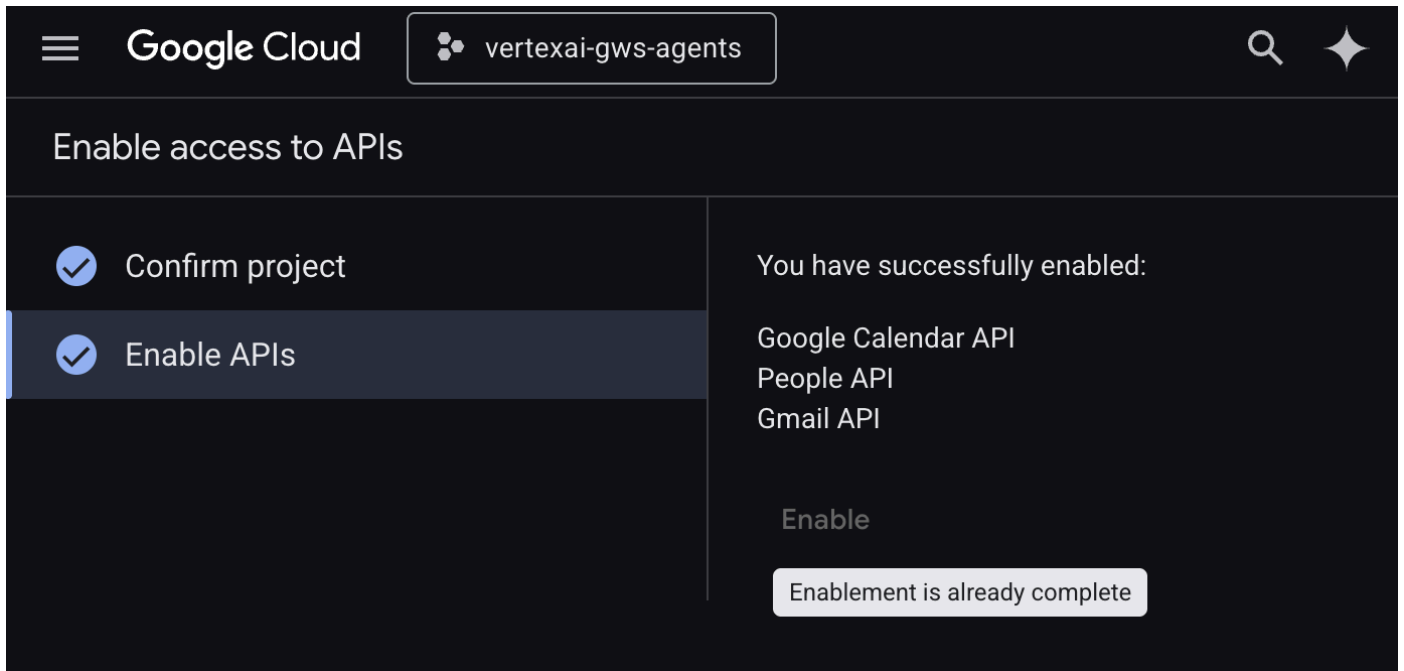
6. 選取「Google Identity」，然後點按「儲存」。



啟用 API

Vertex AI Workspace 資料儲存庫需要啟用下列 API：

1. 在 [Google Cloud 控制台](#) 中，啟用 Calendar、Gmail 和 People API：

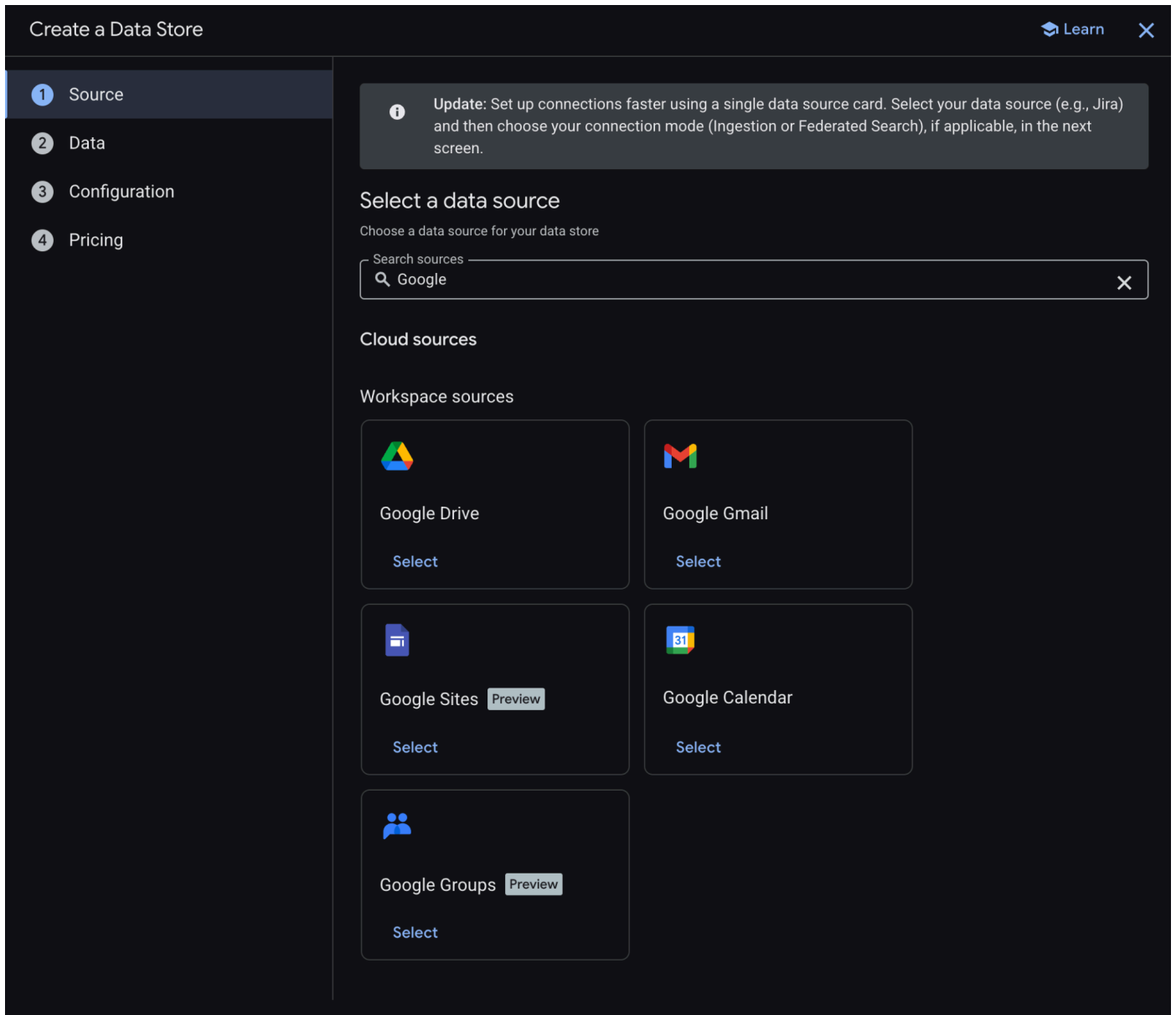


2. 依序點選「選單」圖示 ≡ > 「API 和服務」 > 「已啟用的 API 和服務」，然後確認「Google 日曆 API」、「Gmail API」和「People API」是否在清單中。

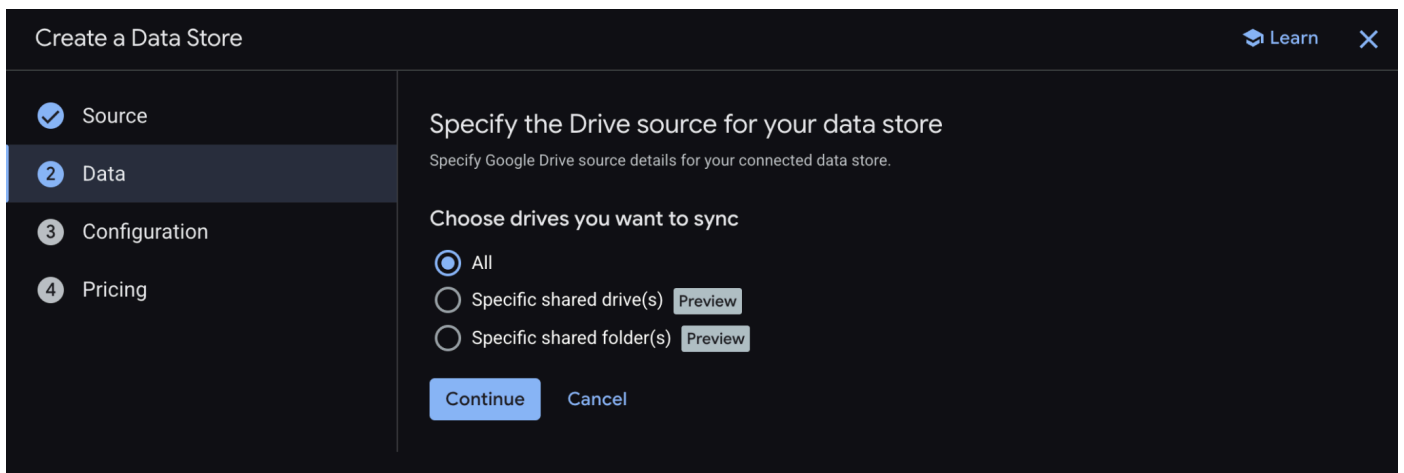
建立資料儲存庫

建立 Google 雲端硬碟資料儲存庫：

1. 前往 Google Cloud 控制台的「AI Applications」，然後前往「Data Stores」。
2. 點選「+ 建立資料儲存庫」。
3. 在「來源」的「Google 雲端硬碟」下方，按一下「選取」。



4. 在「資料」中選取「全部」，然後按一下「繼續」。



5. 在「設定」中，將「資料連接器名稱」設為 `drive`，然後查看並同意可能適用的費用，再點選「繼續」。

Create a Data Store

Learn

Source

Data

Configuration

Pricing

Configure your data connector

Configure additional settings for your data connector

Location of your data connector

Multi-region
global (Global)

Your data connector name

Data connector name *
drive

ID: drive_1772314328791. It cannot be changed later. [Edit](#)

Google Identity will be used for the access control. [Learn more.](#)

The "Smart features in other Google products" setting in Google Workspace must be enabled to search on Workspace data. [Learn more](#)

Charges may apply. To avoid charges, link the data store to a Gemini Enterprise app. [Learn more](#)

Continue

Cancel

6. 在「定價」中，選取偏好的定價模式，然後按一下「建立」。建議您在本程式碼研究室中使用一般定價。

7. 系統會自動將您重新導向至「資料儲存庫」，您可以在這裡查看新加入的資料儲存庫。

建立 Google 日曆資料儲存庫：

- 點選「+ 建立資料儲存庫」。
- 在「來源」中搜尋「Google 日曆」，然後按一下「選取」。
- 在「動作」部分，點按「略過」。
- 在「設定」部分，將「資料連接器名稱」設為 `calendar`。
- 點選「建立」。
- 系統會自動將您重新導向至「資料儲存庫」，您可以在這裡查看新加入的資料儲存庫。

建立 Google Gmail 資料儲存庫：

- 按一下「+ 新增資料儲存庫」。
- 在「Source」(來源) 中搜尋「Google Gmail」，然後按一下「Select」(選取)。
- 在「動作」部分，點按「略過」。
- 在「設定」部分，將「資料連接器名稱」設為 `gmail`。
- 點選「建立」。
- 系統會自動將您重新導向至「資料儲存庫」，您可以在這裡查看新加入的資料儲存庫。

3. Vertex AI Search 應用程式

使用者可以透過自然語言，使用這個代理程式搜尋資料及執行 Workspace 動作。這項功能需要下列元素：

- 模型：Gemini。
- 資料和動作：Google Workspace (日曆、Gmail、雲端硬碟) 的 Vertex AI 資料儲存庫。
- 代理主機：Vertex AI Search。
- 使用者介面：Vertex AI Search 網頁小工具。

複習概念

Vertex AI Search 應用程式

[Vertex AI Search 應用程式](#)會向使用者提供搜尋結果、動作和虛擬服務專員。在 API 的情境中，「應用程式」一詞可與「引擎」互換使用。應用程式必須連結至資料儲存庫，才能使用其中的資料提供搜尋結果、答案或動作。

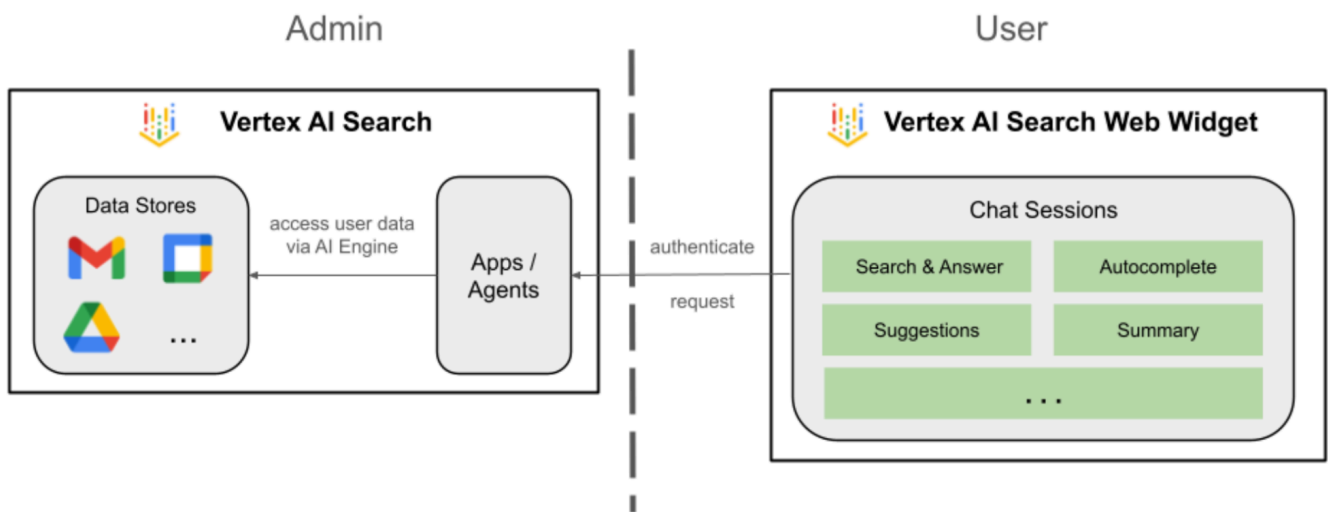
Vertex AI Search 網頁小工具

[Vertex AI Search 網頁小工具](#)是預先建構的可自訂 UI 元件，開發人員只需編寫少量程式碼，即可在網站中直接嵌入 AI 技術輔助搜尋列和結果介面。

Vertex AI Search 預先發布版

Vertex AI Search 預覽版是 Google Cloud 控制台內建的測試環境，可讓開發人員驗證搜尋設定和生成式回覆，然後將這些設定無縫部署至可供正式環境使用的 Vertex AI Search 網頁小工具。

檢閱解決方案架構

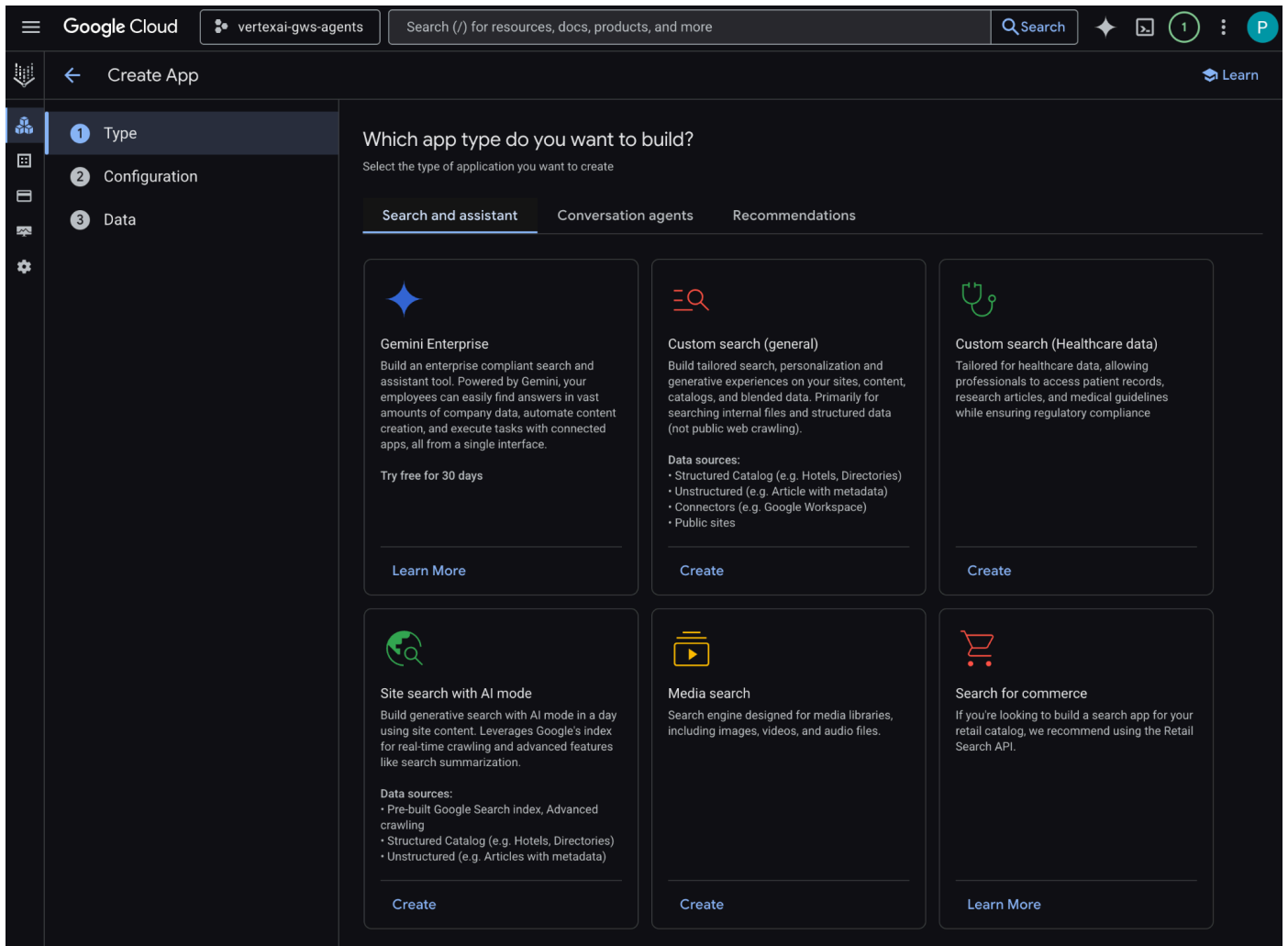


建立應用程式

建立新的搜尋應用程式，做為資料儲存庫的錨點。

從 Cloud 控制台開啟「AI Applications」>「Apps」，然後按照下列步驟操作：

1. 按一下「+ 建立應用程式」。
2. 在「類型」下方的「自訂搜尋 (一般)」，按一下「建立」。



3. 在「設定」中，查看「Enterprise 版功能」和「生成式回覆」，並在確認價格後同意啟用。
4. 將「App name」(應用程式名稱) 設為 `code1ab`。
5. 系統會依據名稱產生 ID，並顯示在欄位下方，請複製該 ID。
6. 將「公司名稱」設為 `Code1ab`。
7. 將「多區域」設為 `global (Global)`。
8. 按一下「繼續」。

Google Cloud

vertexai-gws-agents

Search (/) for resources, docs, products, and more

Search

1

P

Create App

Learn

2 Configuration

3 Data

4 Pricing

Search app configuration

Configure your app settings

☒ Enterprise edition features

In addition to the standard features, you get:

- Extractive answers: answers that are extracted verbatim from your documents.
- Image search, where you can use an image as a query
- Website search
- Core generative answers

Turning on Enterprise edition features is required for website search. To get higher refresh frequency, lower latency, search summaries, and more features in addition to website data, you need to turn on Advanced website indexing. You can change this setting at any time.

After turning on Enterprise features, it can take up to 5 minutes for the features to become available.

[Learn more about features and prices](#)

☒ Generative Responses

For structured, unstructured, and advanced website search, you get:

- Search summarization
- Search with follow-ups
- Advanced generative answers

Generative Responses are not available for basic website search. You can change this setting at any time.

After turning on Generative Responses feature, it can take up to 5 minutes for the features to become available.

[Learn more about features and prices](#)

Your app name

App name *
codelab

ID: codelab_1772313691352. It cannot be changed later. [Edit](#)

External name of your company or organization

Company name *
Codelab

Providing your company name helps the model provide higher-quality responses

Location of your app

We recommend that you choose the global location, if you do not have compliance or regulatory reasons to locate your data in a particular multi-region. (EU and US regions are currently in preview)

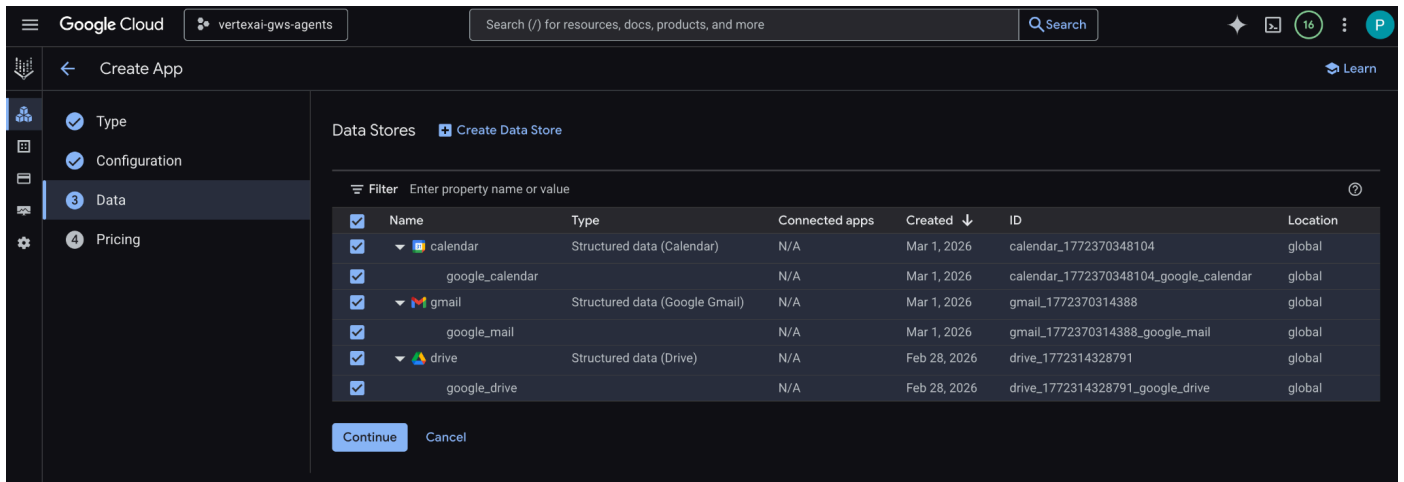
Multi-region *
global (Global)

You can not change it later. For important information about multi-regions, see [Vertex AI Search locations](#)

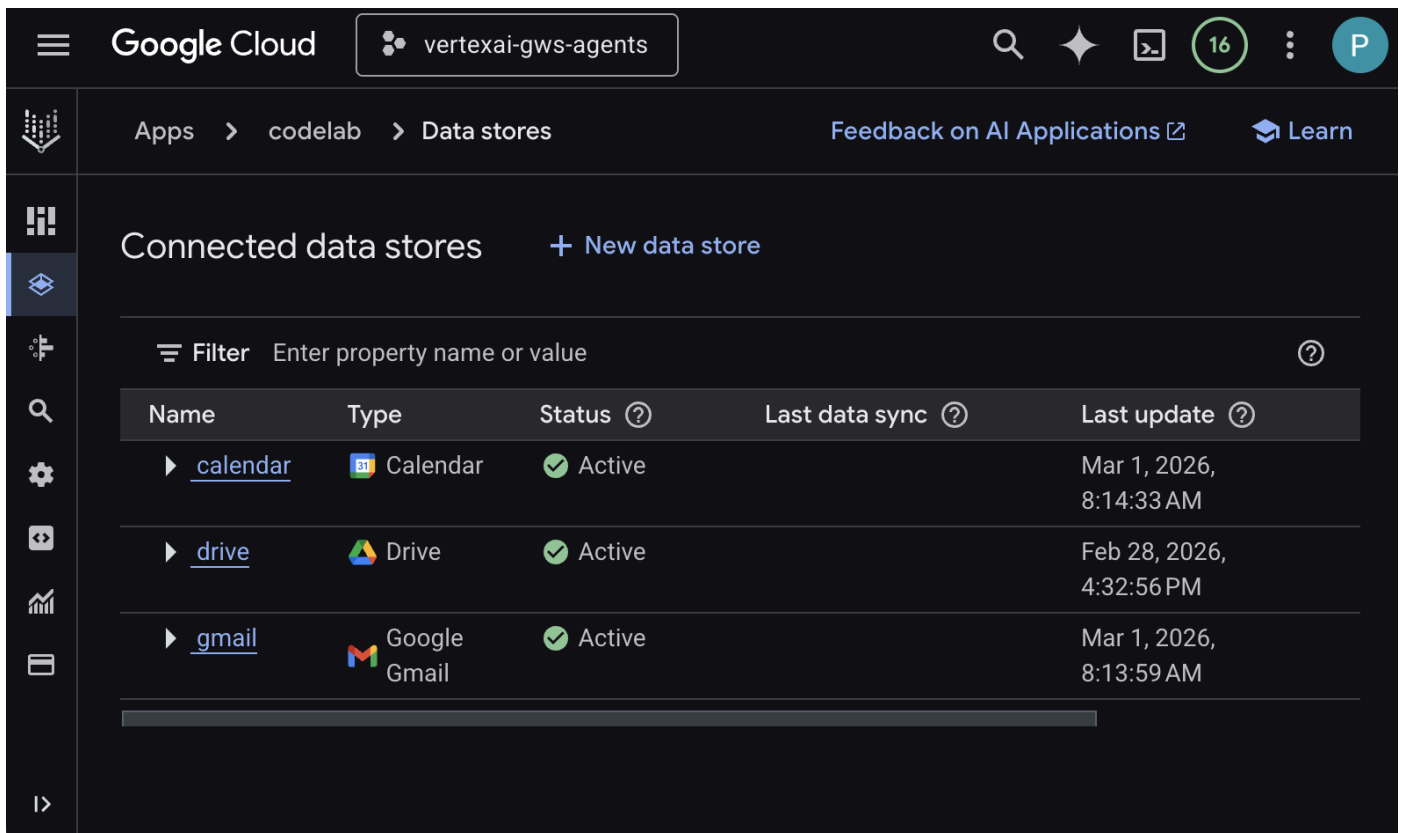
Continue

Cancel

9. 在「資料」中，選取「雲端硬碟」、「Gmail」和「日曆」資料儲存庫，然後按一下「繼續」。



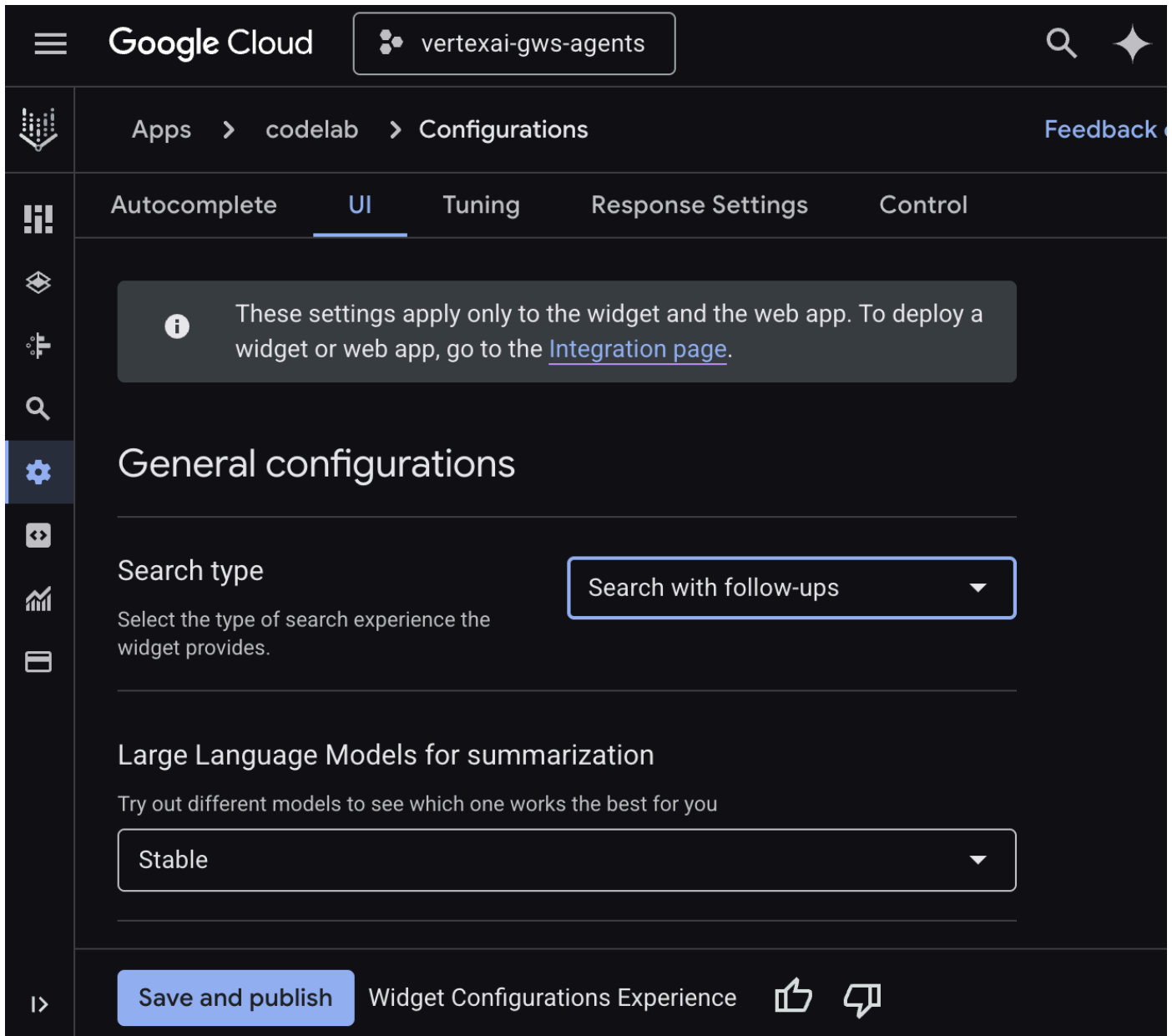
10. 在「定價」中，選取偏好的定價模式，然後按一下「建立」。建議您在本程式碼研究室中使用一般定價。
11. 系統會建立應用程式，並自動重新導向至「AI Applications」>「應用程式」>「codelab」>「應用程式總覽」。
12. 前往「連結的資料儲存庫」。
13. 幾分鐘後，所有已連結的資料儲存庫狀態應會顯示為「Active」(運作中)。



設定 Web Widget

設定搜尋小工具的外觀和行為。

1. 前往「設定」。
2. 在「UI」分頁中，將「搜尋類型」設為「搜尋並追問」，然後按一下「儲存並發布」。



試用應用程式

直接在 Google Cloud 控制台中測試搜尋應用程式。

1. 前往「預覽」，系統會顯示 Web 小工具。
2. 在對話中輸入 `Do I have any meetings today?`，然後按下 `enter` 鍵。
3. 在對話中輸入 `Did I receive an email on March 1st 2026?`，然後按下 `enter` 鍵。
4. 在對話中輸入 `Give me the title of the latest Drive file I created`，然後按下 `enter` 鍵。

Type 

4. 自訂代理

使用者可以透過這個代理程式，使用自訂工具和規則，以自然語言搜尋資料及執行 Workspace 動作。這項功能需要下列元素：

- **模型**：Gemini。
- **資料和動作**：Google Workspace (日曆、Gmail、雲端硬碟) 的 Vertex AI 資料儲存庫、Google 管理的 Vertex AI Search Model Context Protocol (MCP) 伺服器、傳送 Google Chat 訊息的自訂工具函式 (透過 Google Chat API)。
- **代理建構工具**：Agent Development Kit (ADK)。
- **代理主機**：Vertex AI Agent Engine。
- **使用者介面**：ADK Web。

查看概念

Agent Development Kit (ADK)

[Agent Development Kit \(ADK\)](#) 是一套專用工具和框架，提供預先建構的模組，用於推論、記憶體管理和工具整合，簡化自主式 AI 代理的建立程序。

Model Context Protocol (MCP)

[Model Context Protocol \(MCP\)](#) 是一項開放標準，旨在透過通用的「隨插即用」介面，讓 AI 應用程式與各種資料來源或工具無縫安全地整合。

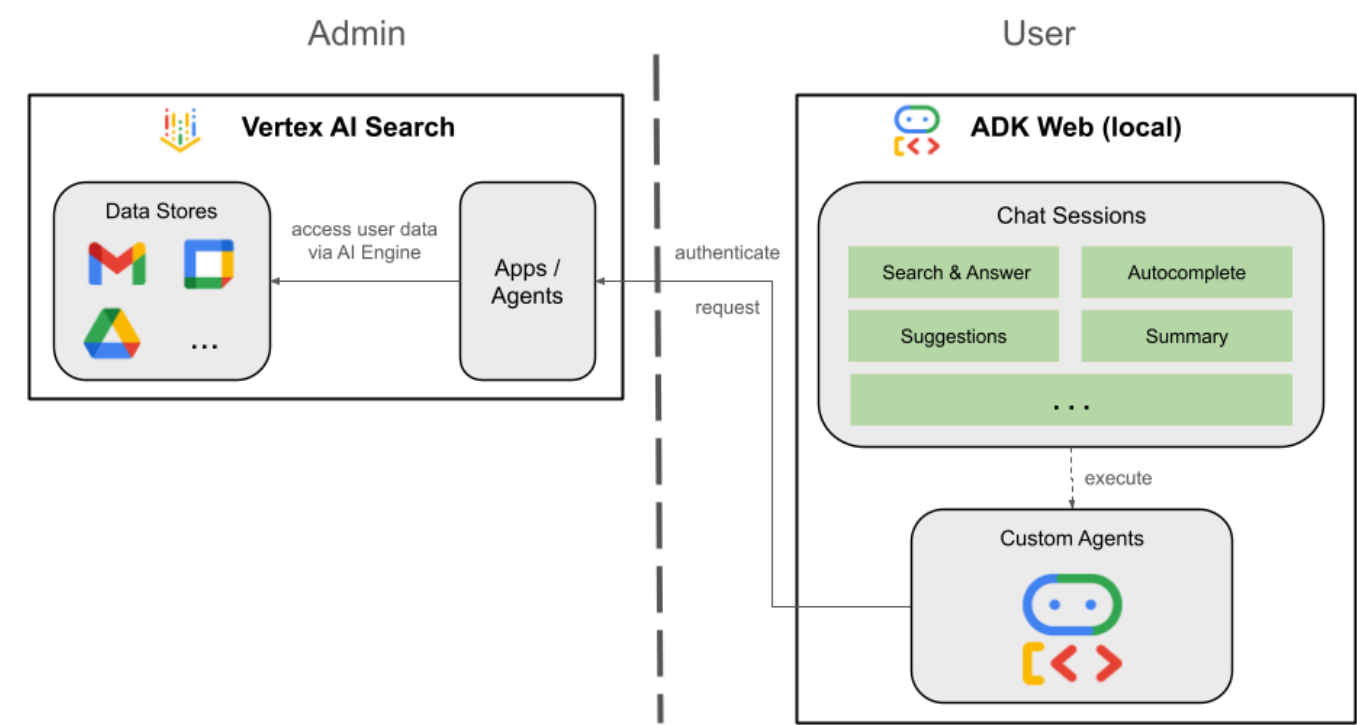
函式工具

[函式工具](#)是預先定義的可執行常式，AI 模型可觸發這項工具來執行特定動作，或從外部系統擷取即時資料，擴充功能，不只是生成文字。

ADK Web

[ADK 網頁](#)是 ADK SDK 隨附的內建開發 UI，可簡化開發和偵錯作業。

查看解決方案架構



查看原始碼

agent.py

下列程式碼會向 Vertex AI 進行驗證、初始化 Vertex AI Search MCP 和 Chat API 工具，並定義代理程式的行為。

- 驗證：**從環境變數擷取 `ACCESS_TOKEN`，驗證 MCP 和 API 呼叫。
- 工具設定：**初始化 `vertexai_mcp`，這個工具集會連線至 Vertex AI Search Model Context Protocol (MCP) 伺服器 and `send_direct_message` 工具。這樣一來，服務專員就能搜尋已連結的資料儲存庫，並傳送 Google Chat 訊息。
- 代理程式定義：**使用 `gemini-2.5-flash` 模型定義 `root_agent`。指令會告知代理優先使用搜尋工具擷取資訊，並使用 `send_direct_message` 工具執行動作，有效讓代理以企業資料為依據。

```

...
MODEL = "gemini-2.5-flash"

# Access token for authentication
ACCESS_TOKEN = os.environ.get("ACCESS_TOKEN")
if not ACCESS_TOKEN:
    raise ValueError("ACCESS_TOKEN environment variable must be set")

VERTEXAI_SEARCH_TIMEOUT = 15.0

def get_project_id():
    """Fetches the consumer project ID from the environment natively."""
    _, project = google.auth.default()
    if project:
        return project
    raise Exception(f"Failed to resolve GCP Project ID from environment.")

def find_serving_config_path():
    """Dynamically finds the default serving config in the engine."""
    project_id = get_project_id()
    engines = discoveryengine_v1.EngineServiceClient().list_engines(
        parent=f"projects/{project_id}/locations/global/collections/default_collection"
    )
    for engine in engines:
        # engine.name natively contains the numeric Project Number
        return f"{engine.name}/servingConfigs/default_serving_config"
    raise Exception(f"No Discovery Engines found in project {project_id}")

def send_direct_message(email: str, message: str) -> dict:
    """Sends a Google Chat Direct Message (DM) to a specific user by email address."""
    chat_client = chat_v1.ChatServiceClient(
        credentials=Credentials(token=ACCESS_TOKEN)
    )

    # 1. Setup the DM space or find existing one
    person = chat_v1.User(
        name=f"users/{email}",
        type_=chat_v1.User.Type.HUMAN
    )
    membership = chat_v1.Membership(member=person)
    space_req = chat_v1.Space(space_type=chat_v1.Space.SpaceType.DIRECT_MESSAGE)
    setup_request = chat_v1.SetUpSpaceRequest(
        space=space_req,
        memberships=[membership]
    )
    space_response = chat_client.set_up_space(request=setup_request)
    space_name = space_response.name

    # 2. Send the message
    msg = chat_v1.Message(text=message)
    message_request = chat_v1.CreateMessageRequest(
        parent=space_name,

```

```

        message=msg
    )
    message_response = chat_client.create_message(request=message_request)

    return {"status": "success", "message_id": message_response.name, "space": space_name}

vertexai_mcp = McpToolset(
    connection_params=StreamableHTTPConnectionParams(
        url="https://discoveryengine.googleapis.com/mcp",
        timeout=VERTEXAI_SEARCH_TIMEOUT,
        sse_read_timeout=VERTEXAI_SEARCH_TIMEOUT,
        headers={"Authorization": f"Bearer {ACCESS_TOKEN}"},
    ),
    tool_filter=['search']
)

# Answer nicely the following user queries:
# - Please find my meetings for today, I need their titles and links
# - What is the latest Drive file I created?
# - What is the latest Gmail message I received?
# - Please send the following message to someone@example.com: Hello, this is a test message.

root_agent = LlmAgent(
    model=MODEL,
    name='enterprise_ai',
    instruction=f"""
        You are a helpful assistant that always uses the Vertex AI MCP search tool to answer
        the user's message, unless the user asks you to send a message to someone.
        If the user asks you to send a message to someone, use the send_direct_message tool
        to send the message.
        You MUST unconditionally use the Vertex AI MCP search tool to find answer, even if
        you believe you already know the answer or believe the Vertex AI MCP search tool does not
        contain the data.
        The Vertex AI MCP search tool accesses the user's data through datastores including
        Google Drive, Google Calendar, and Gmail.
        Only use the Vertex AI MCP search tool with servingConfig and query parameters, do
        not use any other parameters.
        Always use the servingConfig {find_serving_config_path()} while using the Vertex AI
        MCP search tool.
        """,
    tools=[vertexai_mcp, FunctionTool(send_direct_message)]
)

```

下載原始碼

將程式碼範例下載至本機環境，即可開始使用。

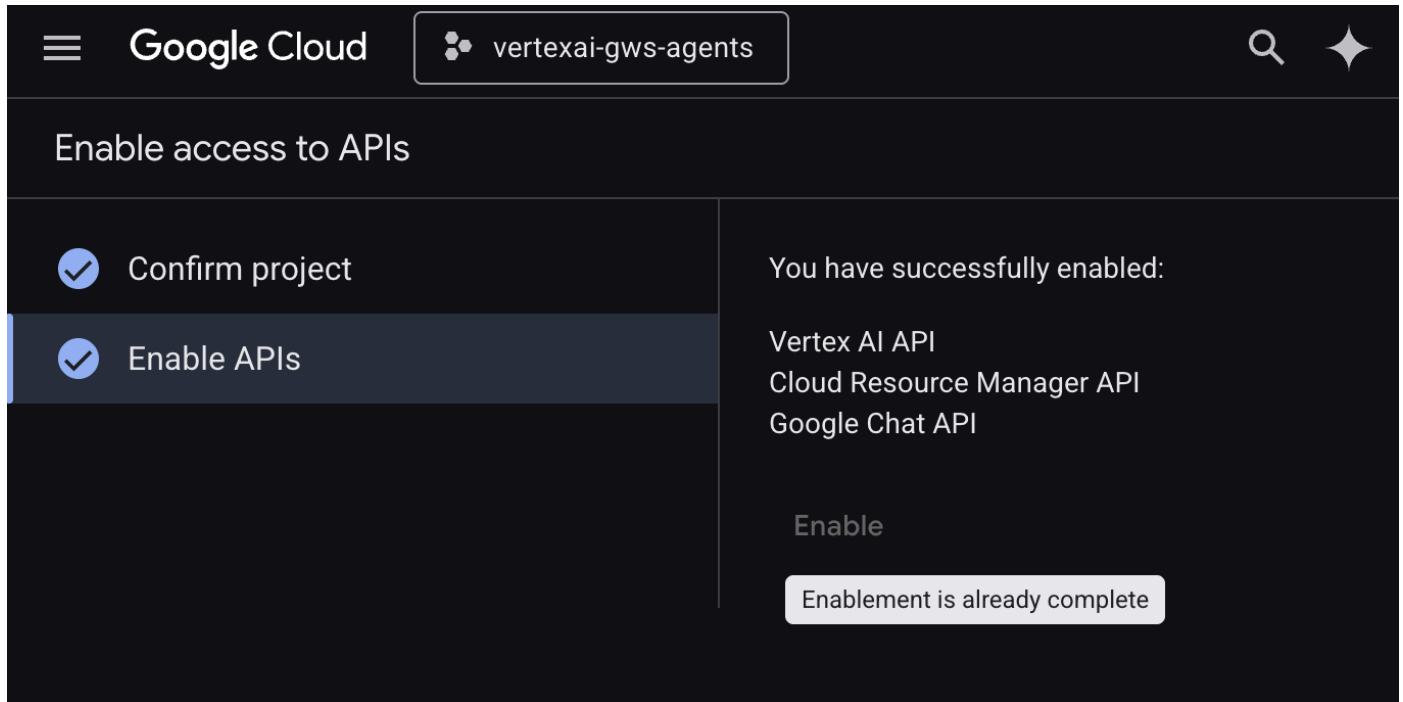
注意：為說明本程式碼研究室的概念，程式碼已大幅簡化。如果您選擇在正式版應用程式中重複使用任何這類程式碼，請務必處理錯誤、徹底測試，並遵循最佳做法。

1. 下載[這個 GitHub 存放區](#)。
2. 在終端機中開啟 `solutions/enterprise-ai-agent-local` 目錄。

啟用 API

這個解決方案需要啟用其他 API：

1. 在 [Google Cloud 控制台](#) 中，啟用 Vertex AI、Cloud Resource Manager 和 Google Chat API：

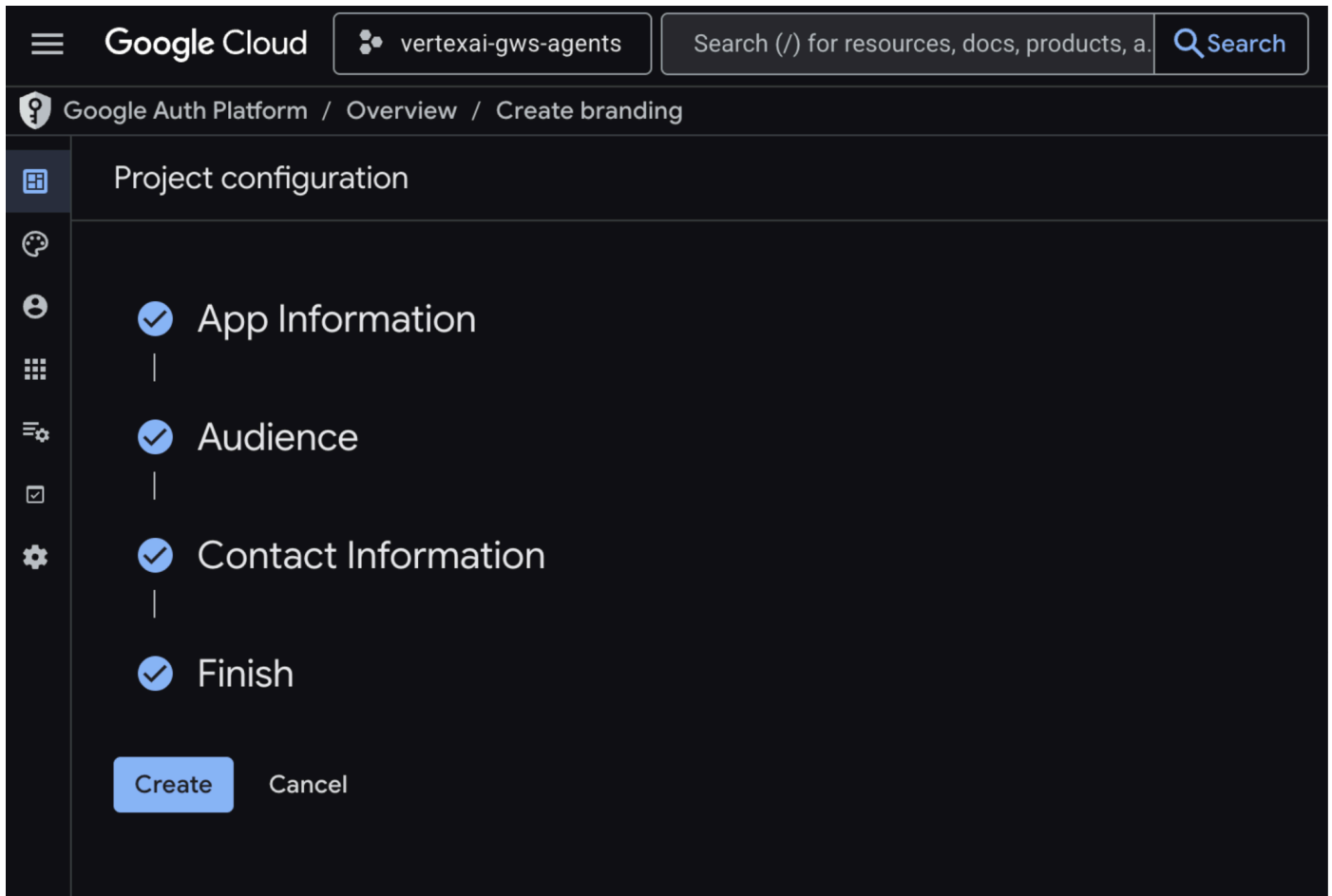


2. 依序點選「選單」圖示 ≡ > 「API 和服務」> 「已啟用的 API 和服務」，然後確認清單中是否列出「Vertex AI API」、「Cloud Resource Manager API」和「Google Chat API」。

設定 OAuth 同意畫面

這個解決方案需要設定同意畫面：

1. 在 [Google Cloud 控制台](#) 中，依序點選「選單」圖示 ≡ > 「Google Auth platform」(Google 驗證平台) > 「Branding」(品牌)。
2. 按一下 [開始使用]。
3. 在「應用程式資訊」下方，將「應用程式名稱」設為 `Codelab`。
4. 在「User support email」(使用者支援電子郵件) 中，選擇支援電子郵件地址，方便使用者在同意聲明方面有任何疑問時與您聯絡。
5. 點選 [下一步]。
6. 在「目標對象」下方，選取「內部」。
7. 點選 [下一步]。
8. 在「聯絡資訊」下方，輸入可接收專案異動通知的電子郵件地址。
9. 點選 [下一步]。
10. 在「完成」部分，請詳閱《[Google API 服務使用者資料政策](#)》，然後選取「我同意《Google API 服務：使用者資料政策》」。
11. 依序點選「繼續」和「建立」。



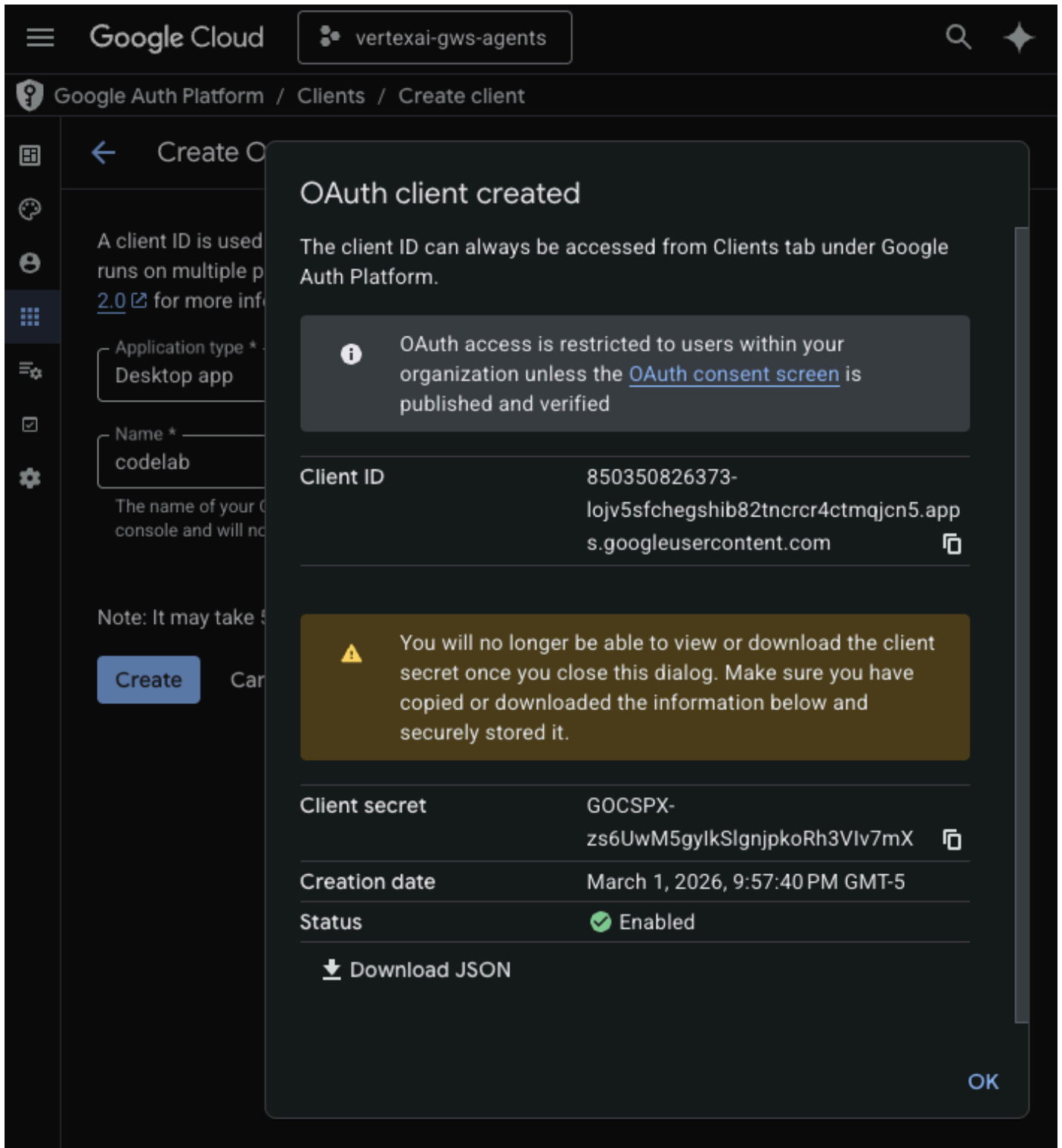
12. 系統會儲存設定，並自動將您重新導向至 **Google Auth Platform > 總覽**。

詳情請參閱完整的「[設定 OAuth 同意畫面](#)」指南。

建立 OAuth 用戶端憑證

建立新的「桌面應用程式」OAuth 用戶端，在本地環境中驗證使用者：

1. 在 [Google Cloud 控制台](#) 中，依序點選「選單」圖示  > 「Google Auth platform」 > 「Clients」。
2. 按一下「+ 建立用戶端」。
3. 在「應用程式類型」中，選取「電腦版應用程式」。
4. 將「Name」(名稱) 設為 `codelab`。
5. 按一下「建立」，系統會顯示新建立的認證。
6. 按一下「Download JSON」(下載 JSON)，然後將檔案儲存為 `client_secret.json`，並放在 `solutions/enterprise-ai-agent-local` 目錄中。



啟用 Vertex AI Search MCP

1. 在終端機中執行：

```
gcloud beta services mcp enable discoveryengine.googleapis.com \
  --project=$(gcloud config get-value project)
```

設定 Chat 擴充應用程式

設定 Google Chat 應用程式的基本資訊詳細資料。

1. 在 [Google Cloud 控制台](#) 的 Google Cloud 搜尋欄位中搜尋 `Google Chat API`，然後依序點選「Google Chat API」、「管理」和「設定」。
2. 將「應用程式名稱」和「說明」設為 `Vertex AI`。
3. 將「Avatar URL」(顯示圖片網址) 設為 `https://developers.google.com/workspace/add-ons/images/quickstart-app-avatar.png`。
4. 取消選取「啟用互動功能」，然後在隨即顯示的模式對話方塊中按一下「停用」。
5. 選取「將錯誤記錄至 Logging」。
6. 按一下 [儲存]。

Google Cloud

vertexai-gws-agents

Search (/) for resources, docs, products.

Search

Google Cloud

API

API/Service Details

Disable API

Metrics

Quotas & System Limits

Credentials

Configuration

Configuration

☒ Build this Chat app as a Workspace add-on.

Chat apps built as Google Workspace add-ons can extend other Google Workspace applications and can be published as one listing with other types of app integrations on the [Google Workspace Marketplace](#). If you clear this checkbox, your Chat app can only work in Google Chat and must be published with a separate Marketplace listing. If you clear this checkbox, you can't revert this setting. For details, see the [developer documentation](#).

Application info

Project number (App ID): 850350826373

App name *

Vertex AI

How the app appears to users who interact with or consume content created by this app, such as in messages, search, and @mentions.

Avatar URL *

https://developers.google.com/workspace/add-ons/images/quickstart-app-a

The app's avatar image. Specify an HTTPS URL that hosts a square (aspect ratio of 1:1) PNG image. Recommended minimum size: 256 by 256 pixels.

Description *

Vertex AI

Max 40 characters

Interactive features

Enable and configure interactive features for a Google Workspace add-on. In Google Chat, add-ons appear to users as Chat apps. [View documentation](#).

Enable Interactive features

Logs

☒ Log errors to Logging

Log Chat app errors to [Logging](#). Debug errors with [Logs Explorer](#).

Save

Cancel

在 ADK Web 中執行代理

使用 ADK 網頁介面在本機啟動代理程式。

- 在終端機中開啟 `solutions/enterprise-ai-agent-local` 目錄，然後執行：

```
# 1. Authenticate with all the required scopes
gcloud auth application-default login \
  --client-id-file=client_secret.json \
  --scopes=https://www.googleapis.com/auth/cloud-
platform,https://www.googleapis.com/auth/chat.spaces,https://www.googleapis.com/auth/
chat.messages

# 2. Configure environment
export ACCESS_TOKEN=$(gcloud auth application-default print-access-token)
export GOOGLE_GENAI_USE_VERTEXAI=1
export GOOGLE_CLOUD_PROJECT=$(gcloud config get-value project)
export GOOGLE_CLOUD_LOCATION=us-central1

# 3. Create and activate a new virtual environment
python3 -m venv .venv
source .venv/bin/activate

# 4. Install poetry and project dependencies
pip install poetry
poetry install

# 5. Start ADK Web
adk web
```

```
+-----+
| ADK Web Server started |
| For local testing, access at http://127.0.0.1:8000. |
+-----+

INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

試用代理

與自訂代理程式對話，驗證流程。

1. 在網際網路瀏覽器中開啟 ADK 網站。
2. 在對話中輸入 `Please find my meetings for today, I need their titles and links`，然後按下 `enter` 鍵。
3. 代理程式會根據使用者帳戶，列出日曆活動。
4. 在對話中輸入 `Please send a Chat message to someone@example.com with the following text: Hello!`，然後按下 `enter` 鍵。
5. 代理程式會回覆確認訊息。



SESSION ID 56fd0b04-827c-4705-a6e9-2d2bf2126ea2

USER ID user



Token Streaming



+ New Session



#1

Please find my meetings for today, I need their titles and links



#2



search

#3



search

#4



Here are your meetings for today:

- **Lunch** Link: <https://www.google.com/calendar/event?eid=N3V2czVpdTN1MTB2cjlxMHQzcDBoYWNmbjAgbWVAcGllcnJpY2t2b3VsZXQuY29t>
- **Important meeting** Link: <https://www.google.com/calendar/event?eid=MGtrNzF2N3Z0ZHRqMjY1cjJOMDUyMzZnMGYgbWVAcGllcnJpY2t2b3VsZXQuY29t>
- **Another meeting** Link: <https://www.google.com/calendar/event?eid=NXRjcGo1aGJ2c25wM2JwMjJhdGwyZXJlZHYgbWVAcGllcnJpY2t2b3VsZXQuY29t>

#5

Please send a Chat message to someone@example.com with the following text: Hello!

#6



send_direct_message

#7



send_direct_message

#8

Your message has been sent to someone@example.com.

Type a Message...



The screenshot displays the Google Chat application. At the top, there's a navigation bar with a search bar and various icons. The main chat window is titled 'someone@example.com' with an 'External' label. The chat history is turned on, and a message 'Hello!' is visible. The bottom of the screen shows a text input area with a 'History is on' status and several quick response buttons: 'What up!', 'I'm here!', and 'Got it!'.

5. 以 Google Workspace 外掛程式形式提供的 Agent

使用者可以在 Workspace 應用程式 UI 中，以自然語言搜尋 Workspace 資料。這項功能需要下列元素：

- **模型**：Gemini。
- **資料和動作**：Google Workspace (日曆、Gmail、雲端硬碟) 的 Vertex AI 資料儲存庫、Google 管理的 Vertex AI Search Model Context Protocol (MCP) 伺服器、傳送 Google Chat 訊息的自訂工具函式 (透過 Google Chat API)。
- **代理建構工具**：Agent Development Kit (ADK)。
- **代理主機**：Vertex AI Agent Engine。
- **使用者介面**：適用於 Chat 和 Gmail 的 Google Workspace 外掛程式 (可輕鬆擴充至 Google 日曆、雲端硬碟、文件、試算表和簡報)。
- **Google Workspace 外掛程式**：Apps Script、Vertex AI Agent Engine API、情境 (選取的 Gmail 郵件)。

複習概念

Google Workspace 外掛程式

[Google Workspace 外掛程式](#)是自訂應用程式，可擴充一或多個 Google Workspace 應用程式 (Gmail、Chat、日曆、文件、雲端硬碟、Meet、試算表和簡報)。

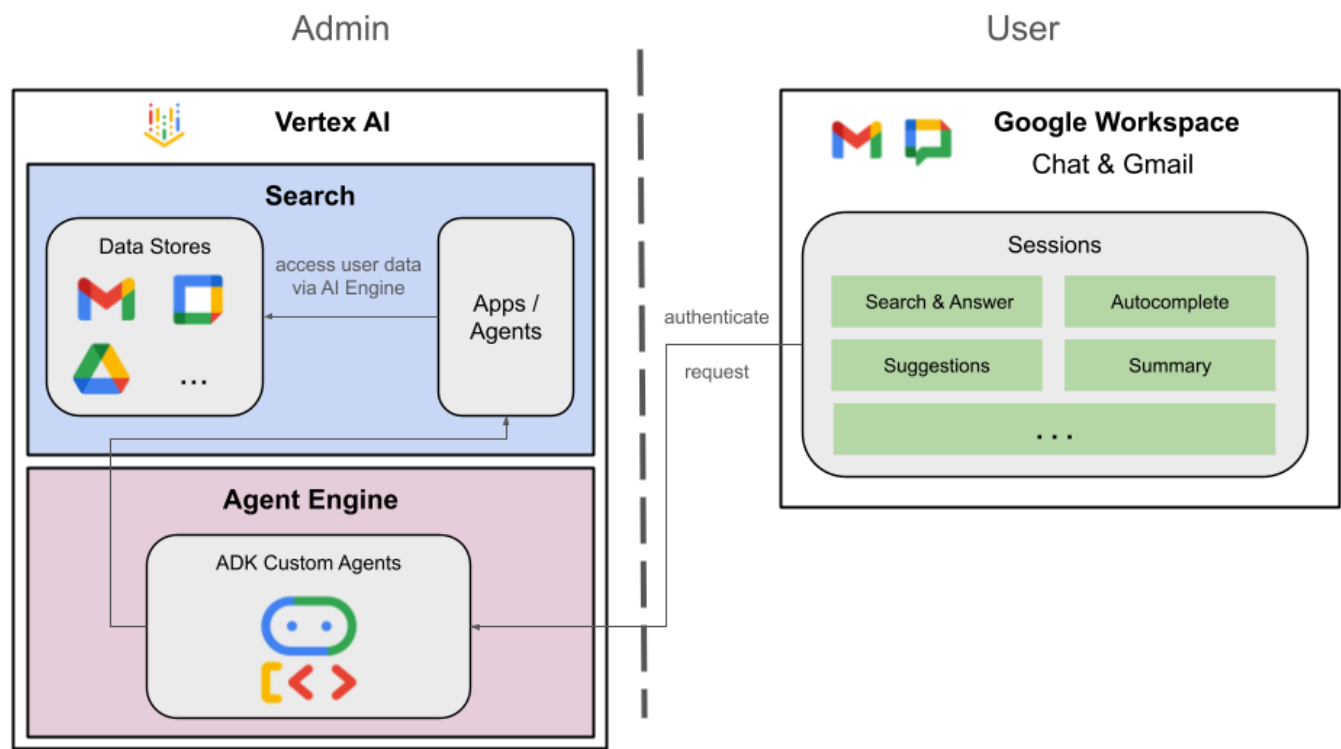
Apps Script

[Apps Script](#) 是以雲端為基礎的 JavaScript 平台，由 Google 雲端硬碟提供支援，可讓您整合及自動執行 Google 產品中的工作。

Google Workspace 資訊卡架構

Google Workspace 的[資訊卡架構](#)可讓開發人員建立豐富的互動式使用者介面。可建構有條理且美觀的資訊卡，當中可包含文字、圖片、按鈕和其他小工具。這些資訊卡提供結構化資訊，並直接在 Workspace 應用程式中啟用快速動作，進而提升使用者體驗。

檢閱解決方案架構



查看原始碼

Agent

agent.py

下列程式碼會向 Vertex AI 進行驗證、初始化 Vertex AI Search MCP 和 Chat API 工具，並定義代理程式的行為。

1. **驗證：**使用輔助函式 `_get_access_token_from_context` 擷取用戶端插入的驗證權杖 (`CLIENT_AUTH_NAME`)。這個權杖對於安全呼叫下游服務 (例如 Vertex AI Search MCP 和 Google Chat 工具) 至關重要。
2. **工具設定：**初始化 `vertexai_mcp`，這個工具集會連線至 Vertex AI Search Model Context Protocol (MCP) 伺服器 and `send_direct_message` 工具。這樣一來，服務專員就能搜尋已連結的資料儲存庫，並傳送 Google Chat 訊息。
3. **代理程式定義：**使用 `gemini-2.5-flash` 模型定義 `root_agent`。指令會告知代理優先使用搜尋工具擷取資訊，並使用 `send_direct_message` 工具執行動作，有效讓代理以企業資料為依據。

```

...
MODEL = "gemini-2.5-flash"

# Client injects a bearer token into the ToolContext state.
# The key pattern is "CLIENT_AUTH_NAME_<random_digits>".
# We dynamically parse this token to authenticate our MCP and API calls.
CLIENT_AUTH_NAME = "enterprise-ai"

VERTEXAI_SEARCH_TIMEOUT = 15.0

def get_project_id():
    """Fetches the consumer project ID from the environment natively."""
    _, project = google.auth.default()
    if project:
        return project
    raise Exception(f"Failed to resolve GCP Project ID from environment.")

def find_serving_config_path():
    """Dynamically finds the default serving config in the engine."""
    project_id = get_project_id()
    engines = discoveryengine_v1.EngineServiceClient().list_engines(
        parent=f"projects/{project_id}/locations/global/collections/default_collection"
    )
    for engine in engines:
        # engine.name natively contains the numeric Project Number
        return f"{engine.name}/servingConfigs/default_serving_config"
    raise Exception(f"No Discovery Engines found in project {project_id}")

def _get_access_token_from_context(tool_context: ToolContext) -> str:
    """Helper method to dynamically parse the intercepted bearer token from the context state."""
    escaped_name = re.escape(CLIENT_AUTH_NAME)
    pattern = re.compile(fr"^{escaped_name}_\d+$")
    # Handle ADK varying state object types (Raw Dict vs ADK State)
    state_dict = tool_context.state.to_dict() if hasattr(tool_context.state, 'to_dict') else tool_context.state
    matching_keys = [k for k in state_dict.keys() if pattern.match(k)]
    if matching_keys:
        return state_dict.get(matching_keys[0])
    raise Exception(f"No bearer token found in ToolContext state matching pattern {pattern.pattern}")

def auth_header_provider(tool_context: ToolContext) -> dict[str, str]:
    token = _get_access_token_from_context(tool_context)
    return {"Authorization": f"Bearer {token}"}

def send_direct_message(email: str, message: str, tool_context: ToolContext) -> dict:
    """Sends a Google Chat Direct Message (DM) to a specific user by email address."""
    chat_client = chat_v1.ChatServiceClient(
        credentials=Credentials(token=_get_access_token_from_context(tool_context))
    )

```

```

# 1. Setup the DM space or find existing one
person = chat_v1.User(
    name=f"users/{email}",
    type_=chat_v1.User.Type.HUMAN
)
membership = chat_v1.Membership(member=person)
space_req = chat_v1.Space(space_type=chat_v1.Space.SpaceType.DIRECT_MESSAGE)
setup_request = chat_v1.SetUpSpaceRequest(
    space=space_req,
    memberships=[membership]
)
space_response = chat_client.set_up_space(request=setup_request)
space_name = space_response.name

# 2. Send the message
msg = chat_v1.Message(text=message)
message_request = chat_v1.CreateMessageRequest(
    parent=space_name,
    message=msg
)
message_response = chat_client.create_message(request=message_request)

return {"status": "success", "message_id": message_response.name, "space": space_name}

vertexai_mcp = McpToolset(
    connection_params=StreamableHTTPConnectionParams(
        url="https://discoveryengine.googleapis.com/mcp",
        timeout=VERTEXAI_SEARCH_TIMEOUT,
        sse_read_timeout=VERTEXAI_SEARCH_TIMEOUT
    ),
    tool_filter=['search'],
    # The auth_header_provider dynamically injects the bearer token from the ToolContext
    # into the MCP call for authentication.
    header_provider=auth_header_provider
)

# Answer nicely the following user queries:
# - Please find my meetings for today, I need their titles and links
# - What is the latest Drive file I created?
# - What is the latest Gmail message I received?
# - Please send the following message to someone@example.com: Hello, this is a test message.

root_agent = LlmAgent(
    model=MODEL,
    name='enterprise_ai',
    instruction=f"""
        You are a helpful assistant that always uses the Vertex AI MCP search tool to answer
        the user's message, unless the user asks you to send a message to someone.
        If the user asks you to send a message to someone, use the send_direct_message tool
        to send the message.
        You MUST unconditionally use the Vertex AI MCP search tool to find answer, even if
        you believe you already know the answer or believe the Vertex AI MCP search tool does not
    """
)

```

contain the data.

The Vertex AI MCP search tool accesses the user's data through datastores including Google Drive, Google Calendar, and Gmail.

Only use the Vertex AI MCP search tool with `servingConfig` and `query` parameters, do not use any other parameters.

Always use the `servingConfig {find_serving_config_path()}` while using the Vertex AI MCP search tool.

```
""",
tools=[vertexai_mcp, FunctionTool(send_direct_message)]
)
```

客戶

appscript.json

下列設定定義外掛程式的觸發條件和權限。

1. **定義外掛程式**：向 Workspace 說明這個專案是 **Chat** 和 **Gmail** 的外掛程式。
2. **內容比對觸發條件**：針對 Gmail，這項設定會建立 `contextualTrigger`，每當使用者開啟電子郵件訊息時，就會觸發 `onAddonEvent`。外掛程式就能「查看」電子郵件內容。
3. **權限**：列出外掛程式執行所需的 `oauthScopes`，例如讀取目前電子郵件、執行指令碼，以及連線至外部服務 (如 Vertex AI API) 的權限。

```
...
"addOns": {
  "common": {
    "name": "Vertex AI",
    "logoUrl": "https://developers.google.com/workspace/add-ons/images/quickstart-app-
avatar.png"
  },
  "chat": {},
  "gmail": {
    "contextualTriggers": [
      {
        "unconditional": {},
        "onTriggerFunction": "onAddonEvent"
      }
    ]
  }
},
"oauthScopes": [
  "https://www.googleapis.com/auth/script.external_request",
  "https://www.googleapis.com/auth/cloud-platform",
  "https://www.googleapis.com/auth/gmail.addons.execute",
  "https://www.googleapis.com/auth/gmail.addons.current.message.readonly"
]
...
```

Chat.gs

下列程式碼會處理傳入的 Google Chat 訊息。

1. **接收訊息**： `onMessage` 函式是訊息互動的進入點。
2. **管理內容**：將 `space.name` (Chat 聊天室的 ID) 儲存至使用者的屬性。這樣一來，代理程式準備好回覆時，就能確切知道要將訊息發布到哪個對話。
3. **委派給代理程式**：呼叫 `requestAgent`，將使用者訊息傳遞至處理 API 通訊的核心邏輯。

```

...
// Service that handles Google Chat operations.

// Handle incoming Google Chat message events, actions will be taken via Google Chat API
calls
function onMessage(event) {
  if (isInDebugMode()) {
    console.log(`MESSAGE event received (Chat): ${JSON.stringify(event)}`);
  }
  // Extract data from the event.
  const chatEvent = event.chat;
  setChatConfig(chatEvent.messagePayload.space.name);

  // Request AI agent to answer the message
  requestAgent(chatEvent.messagePayload.message);
  // Respond with an empty response to the Google Chat platform to acknowledge execution
  return null;
}

// --- Utility functions ---

// The Chat direct message (DM) space associated with the user
const SPACE_NAME_PROPERTY = "DM_SPACE_NAME"

// Sets the Chat DM space name for subsequent operations.
function setChatConfig(spaceName) {
  const userProperties = PropertiesService.getUserProperties();
  userProperties.setProperty(SPACE_NAME_PROPERTY, spaceName);
  console.log(`Space is set to ${spaceName}`);
}

// Retrieved the Chat DM space name to sent messages to.
function getConfiguredChat() {
  const userProperties = PropertiesService.getUserProperties();
  return userProperties.getProperty(SPACE_NAME_PROPERTY);
}

// Finds the Chat DM space name between the Chat app and the given user.
function findChatAppDm(userName) {
  return Chat.Spaces.findDirectMessage(
    { 'name': userName },
    { 'Authorization': `Bearer ${getAddonCredentials().getAccessToken()}` }
  ).name;
}

// Creates a Chat message in the configured space.
function createMessage(message) {
  const spaceName = getConfiguredChat();
  console.log(`Creating message in space ${spaceName}...`);
  return Chat.Spaces.Messages.create(
    message,
    spaceName,

```

```
    {} ,  
    { 'Authorization': `Bearer ${getAddonCredentials().getAccessToken()}` }  
  ).name;  
}
```

Sidebar.gs

下列程式碼會建構 Gmail 側欄，並擷取電子郵件內容。

1. **建構 UI**： `createSidebarCard` 使用 `Workspace` 資訊卡服務建構視覺化介面。這個範本會建立簡單的版面配置，包含文字輸入區和「傳送訊息」按鈕。
2. **擷取電子郵件內容**：在 `handleSendMessage` 中，程式碼會檢查使用者目前正在查看電子郵件 (`event.gmail.messageId`)。如果是，程式碼會安全地擷取電子郵件的主旨和內文，並附加至使用者的提示。
3. **顯示結果**：代理程式回覆後，程式碼會更新側邊欄資訊卡，顯示回覆內容。


```

...
// Service that handles Gmail operations.

// Triggered when the user opens the Gmail Add-on or selects an email.
function onAddonEvent(event) {
  // If this was triggered by a button click, handle it
  if (event.parameters && event.parameters.action === 'send') {
    return handleSendMessage(event);
  }

  // Otherwise, just render the default initial sidebar
  return createSidebarCard();
}

// Creates the standard Gmail sidebar card consisting of a text input and send button.
// Optionally includes an answer section if a response was generated.
function createSidebarCard(optionalAnswerSection) {
  const card = CardService.newCardBuilder();
  const actionSection = CardService.newCardSection();

  // Create text input for the user's message
  const messageInput = CardService.newTextInput()
    .setFieldName("message")
    .setTitle("Message")
    .setMultiline(true);

  // Create action for sending the message
  const sendAction = CardService.newAction()
    .setFunctionName('onAddonEvent')
    .setParameters({ 'action': 'send' });

  const sendButton = CardService.newTextButton()
    .setText("Send message")
    .setTextButtonStyle(CardService.TextButtonStyle.FILLED)
    .setOnClickAction(sendAction);

  actionSection.addWidget(messageInput);
  actionSection.addWidget(CardService.newButtonSet().addButton(sendButton));

  card.addSection(actionSection);

  // Attach the response at the bottom if we have one
  if (optionalAnswerSection) {
    card.addSection(optionalAnswerSection);
  }

  return card.build();
}

// Handles clicks from the Send message button.
function handleSendMessage(event) {
  const commonEventObject = event.commonEventObject || {};

```

```

const formInputs = commonEventObject.formInputs || {};
const messageInput = formInputs.message;

let userMessage = "";
if (messageInput && messageInput.stringInputs && messageInput.stringInputs.value.length >
0) {
    userMessage = messageInput.stringInputs.value[0];
}

if (!userMessage || userMessage.trim().length === 0) {
    return CardService.newActionResponseBuilder()
        .setNotification(CardService.newNotification().setText("Please enter a message."))
        .build();
}

let finalQueryText = `USER MESSAGE TO ANSWER: ${userMessage}`;

// If we have an email selected in Gmail, append its content as context
if (event.gmail && event.gmail.messageId) {
    try {
        GmailApp.setCurrentMessageAccessToken(event.gmail.accessToken);
        const message = GmailApp.getMessageById(event.gmail.messageId);

        const subject = message.getSubject();
        const bodyText = message.getPlainBody() || message.getBody();

        finalQueryText += `\n\nEMAIL THE USER HAS OPENED ON SCREEN:\nSubject:
${subject}\nBody:\n---\n${bodyText}\n---`;
    } catch (e) {
        console.error("Could not fetch Gmail context: " + e);
        // Invalidate the token explicitly so the next prompt requests the missing scopes
        ScriptApp.invalidateAuth();

        CardService.newAuthorizationException()
            .setResourceDisplayName("Enterprise AI")
            .setAuthorizationUrl(ScriptApp.getAuthorizationUrl())
            .throwException();
    }
}

try {
    const response = queryAgent({ text: finalQueryText });

    // We leverage the 'showdown' library to parse the LLM's Markdown output into HTML
    // We also substitute markdown listings with arrows and adjust newlines for clearer
rendering in the sidebar
    let displayedText = substituteListingsFromMarkdown(response.text);
    displayedText = new showdown.Converter().makeHtml(displayedText).replace(/\n/g, '\n\n');

    const textParagraph = CardService.newTextParagraph();
    textParagraph.setText(displayedText);

```

```

const answerSection = CardService.newCardSection()
  .addWidget(textParagraph);

const updatedCard = createSidebarCard(answerSection);

return CardService.newActionResponseBuilder()
  .setNavigation(CardService.newNavigation().updateCard(updatedCard))
  .build();

} catch (err) {
  return CardService.newActionResponseBuilder()
    .setNotification(CardService.newNotification().setText("Error fetching response: " +
err.message))
    .build();
}
}
...

```

AgentHandler.gs

下列程式碼會協調對 Vertex AI 的 API 呼叫。

1. **自動調度 API 呼叫**： `queryAgent` 是外掛程式與 Vertex AI Agent Engine 之間的橋樑。這會建構要求，其中包含使用者的查詢和狀態中的驗證權杖。
2. **串流回應**：由於代理程式回應可能需要一些時間，因此會搭配伺服器傳送事件 (SSE) 使用 `streamQuery` API。程式碼會分批收集回覆，並重構完整答案。

```

...
// Service that handles Vertex AI Agent operations.

// Submits a query to the AI agent and returns the response string synchronously
function queryAgent(input) {
  let systemPrompt = "SYSTEM PROMPT START Do not respond with tables but use bullet points
instead." +
    " Do not ask the user follow-up questions or converse with them as history is not kept in
this interface." +
    " SYSTEM PROMPT END\n\n";

  const requestPayload = {
    "class_method": "async_stream_query",
    "input": {
      "user_id": "vertex_ai_add_on",
      "message": { "role": "user", "parts": [{ "text": systemPrompt + input.text }] },
      "state_delta": {
        "enterprise-ai_999": `${ScriptApp.getOAuthToken()}`
      }
    }
  };

  const responseContentText = UrlFetchApp.fetch(
    `https://${getLocation()}-
aiplatform.googleapis.com/v1/${getReasoningEngine()}:streamQuery?alt=sse`,
    {
      method: 'post',
      headers: { 'Authorization': `Bearer ${ScriptApp.getOAuthToken()}` },
      contentType: 'application/json',
      payload: JSON.stringify(requestPayload),
      muteHttpExceptions: true
    }
  ).getContentText();
  if (isInDebugMode()) {
    console.log(`Response: ${responseContentText}`);
  }

  const events = responseContentText.split('\n').map(s => s.replace(/^data:\s*/, '')).filter(s
=> s.trim().length > 0);
  console.log(`Received ${events.length} agent events.`);

  let author = "default";
  let answerText = "";
  for (const eventJson of events) {
    if (isInDebugMode()) {
      console.log("Event: " + eventJson);
    }
    const event = JSON.parse(eventJson);

    // Retrieve the agent responsible for generating the content
    author = event.author;

```

```
// Ignore events that are not useful for the end-user
if (!event.content) {
  console.log(`${author}: internal event`);
  continue;
}

// Handle text answers
const parts = event.content.parts || [];
const textPart = parts.find(p => p.text);
if (textPart) {
  answerText += textPart.text;
}
}
return { author: author, text: answerText };
}
...
```

在 Vertex AI Agent Engine 部署代理

1. 在終端機中，開啟從先前步驟下載來源的 `solutions/enterprise-ai-agent` 目錄，然後執行：

```
# 1. Create and activate a new virtual environment
deactivate
python3 -m venv .venv
source .venv/bin/activate

# 2. Install poetry and project dependencies
pip install poetry
poetry install

# 3. Deploy the agent
adk deploy agent_engine \
  --project=$(gcloud config get-value project) \
  --region=us-central1 \
  --display_name="Enterprise AI" \
  enterprise_ai
```

```
Copying agent source code...
Copying agent source code complete.
Resolving files and dependencies...
Initializing Vertex AI...
Vertex AI initialized.
Created enterprise_ai_tmp20260223_170118/agent_engine_app.py
Files and dependencies resolved
Deploying to agent engine...
```

2. 在記錄中看到「Deploying to agent engine...」這行時，請開啟新的終端機並執行下列指令，將必要權限新增至 **Vertex AI Reasoning Engine** 服務代理程式：

```
# 1. Get the current Project ID
PROJECT_ID=$(gcloud config get-value project)

# 2. Extract the Project Number for that ID
PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --
format='value(projectNumber)')

# 3. Construct the Service Account name
SERVICE_ACCOUNT="service-${PROJECT_NUMBER}@gcp-sa-aiplatform-
re.iam.gserviceaccount.com"

# 4. Apply the IAM policy binding
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$SERVICE_ACCOUNT" \
  --role="roles/discoveryengine.viewer"
```

3. 等待 **adk deploy** 指令執行完畢，然後從指令輸出內容中複製以綠色標示的新部署代理資源名稱。

```
Copying agent source code...
Copying agent source code complete.
Resolving files and dependencies...
Initializing Vertex AI...
Vertex AI initialized.
Created enterprise_ai_tmp20260223_170118/agent_engine_app.py
Files and dependencies resolved
Deploying to agent engine...
✅ Updated agent engine: projects/ge-gws-agents-lab/locations/us-central1/reasoningEngines/998377448741535744
Cleaning up the temp folder: enterprise_ai tmp20260223_170118
(.venv) [LOAS2] pierrick@pierrick:~/git/python-samples/solutions/enterprise-ai-agent (enterprise-ai) $
```

啟動服務帳戶

建立專用服務帳戶，授權外掛程式的伺服器端作業。

在 [Google Cloud 控制台](#) 中，按照下列步驟操作：

1. 依序點選「選單 ≡」>「IAM 與管理」>「服務帳戶」>「+ 建立服務帳戶」。
2. 將「服務帳戶名稱」設為 `vertexai-add-on`。

Google Cloud

vertexai-gws-agents

4

P

IAM & Admin / Service accounts / Create service account

Create service account

1

Create service account

Service account name

vertexai-add-on

Display name for this service account

Service account ID *

vertexai-add-on

X

↺

Email address: vertexai-add-on@vertexai-gws-agents-489002.iam.gserviceaccount.com

Service account description

Describe what this service account will do

Create and continue

2

Permissions (optional)

3

Principals with access (optional)

Done

Cancel

3. 按一下「完成」，系統會將您重新導向至「服務帳戶」頁面，您可以在該頁面中看到建立的服務帳戶。

Google Cloud

vertexai-gws-agents

Search (/) for resources, docs, products, and more

IAM & Admin / Service accounts

Service accounts

+ Create service account

Delete

Manage access

Refresh

Service accounts for project "vertexai-gws-agents"

A service account represents a Google Cloud service identity, such as code running on Compute Engine VMs, App Engine apps,

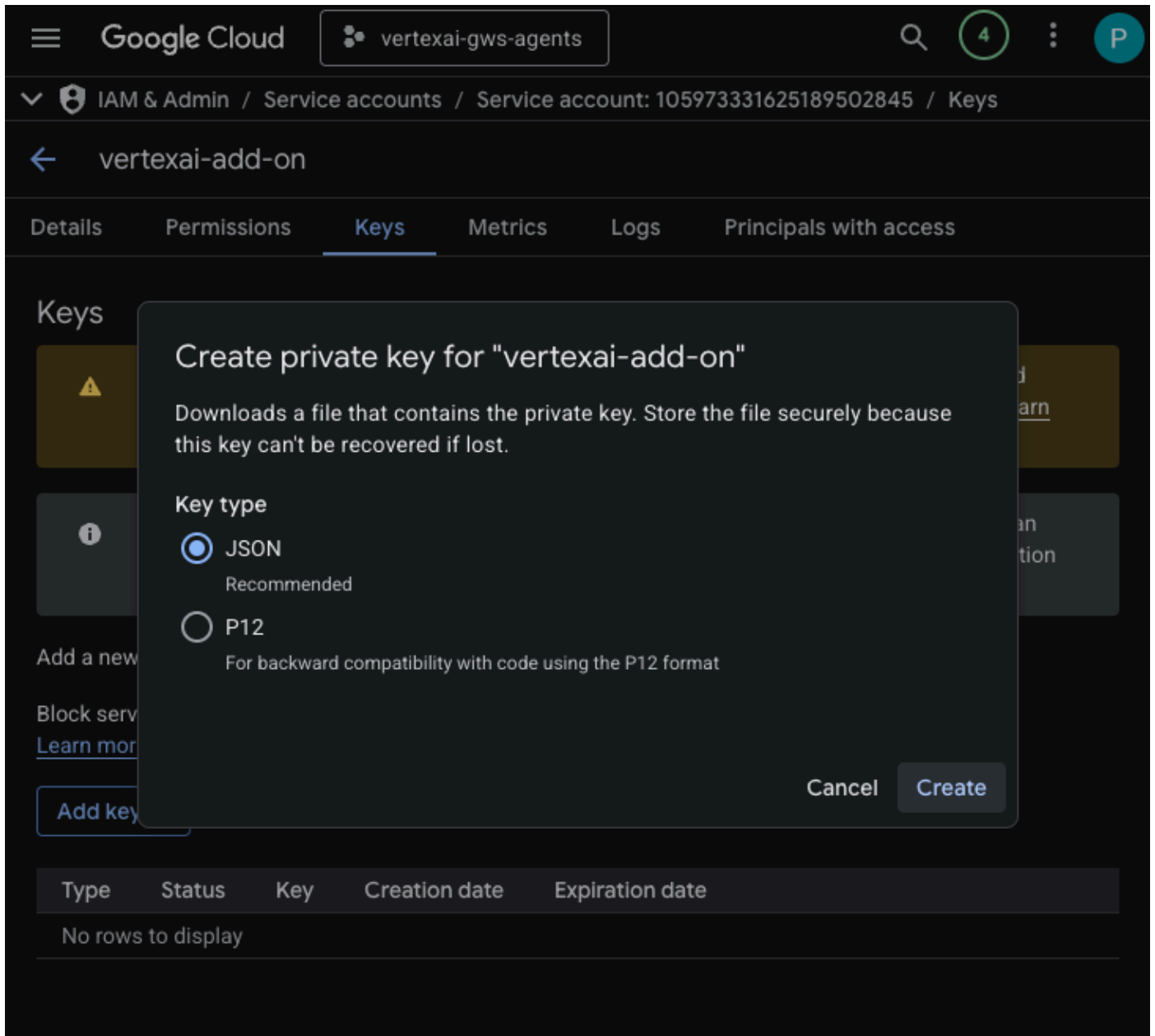
Organization policies can be used to secure service accounts and block risky service account features, such as automatic IAM [service account organization policies](#).

Filter

Enter property name or value

☐	Email	Status	Name ↑	Description
☐	vertexai-add-on@vertexai-gws-agents-489002.iam.gserviceaccount.com	Enabled	vertexai-add-on	

4. 選取新建立的服務帳戶，然後選取「金鑰」分頁標籤。
5. 依序點選「新增金鑰」和「建立新的金鑰」。
6. 選取「JSON」，然後按一下「建立」。



7. 對話方塊會關閉，新建立的公開/私密金鑰組會自動以 JSON 檔案的形式下載至本機環境。

建立及設定 Apps Script 專案

建立新的 Apps Script 專案來託管外掛程式程式碼，並設定連線屬性。

注意：為說明本程式碼研究室的概念，程式碼已大幅簡化。如果您選擇在正式版應用程式中重複使用任何這類程式碼，請務必處理錯誤、徹底測試，並遵循最佳做法。

- 點選下列按鈕，開啟 **Enterprise AI 外掛程式** Apps Script 專案：
- 依序點選「總覽」>「建立副本」。
- 在 Apps Script 專案中，依序點選「專案設定」>「編輯指令碼屬性」>「新增指令碼屬性」，即可新增指令碼屬性。
- 將 **REASONING_ENGINE_RESOURCE_NAME** 設為先前步驟中複製的 Vertex AI 代理程式資源名稱。格式如下：

```
projects/<PROJECT_NUMBER>/locations/us-central1/reasoningEngines/<AGENT_ID>
```

5. 將 **APP_SERVICE_ACCOUNT_KEY** 設為先前步驟下載的服務帳戶檔案中的 JSON 金鑰。

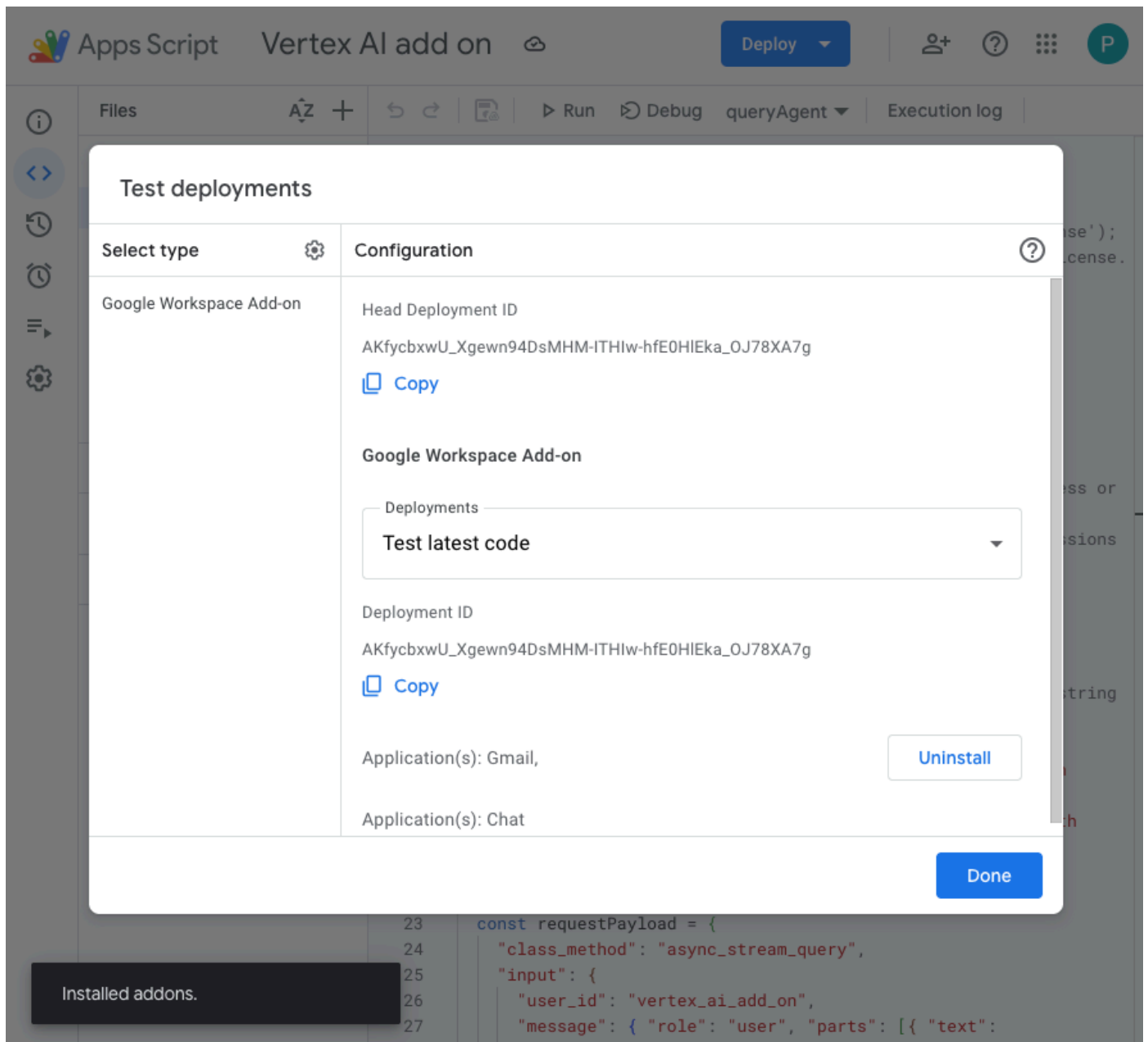
6. 按一下「儲存指令碼屬性」

部署至 Gmail 和 Chat

部署外掛程式，直接在 Gmail 和 Google Chat 中測試。

在 Apps Script 專案中，請按照下列步驟操作：

- 依序點選「部署」>「測試部署作業」，然後點選「安裝」。這項功能現已在 Gmail 中推出。
- 按一下「首要部署作業 ID」下方的「複製」。

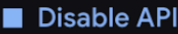


在 [Google Cloud 控制台](#) 中，按照下列步驟操作：

- 在 Google Cloud 搜尋欄位中搜尋 **Google Chat API**，依序點選「Google Chat API」、「管理」和「設定」。

2. 選取「啟用互動功能」。
3. 取消選取「加入聊天室和群組對話」。
4. 在「連線設定」下方，選取「Apps Script」。
5. 將「部署作業 ID」設為上一個步驟中複製的「首要部署作業 ID」。
6. 在「瀏覽權限」下方，選取「將這個 Chat 應用程式提供給 Your Workspace Domain 中的特定使用者和群組」，然後輸入電子郵件地址。
7. 按一下 [儲存]。

 vertexai-gws-agents  

API  API/Service Details 

 Configuration



Interactive features

Enable and configure interactive features for a Google Workspace add-on. In Google Chat, add-ons appear to users as Chat apps. [View documentation](#).

☒ Enable Interactive features

Functionality

Users can find and message the app directly in Google Chat. When a user sends the app a direct message, it receives a MESSAGE [event](#).

☐ Join spaces and group conversations
The app can join spaces and group conversations by getting added to them, receiving an ADDED_TO_SPACE [event](#).

Connection settings

Specify a deployment for your Chat app. For guidance, view the [documentation](#).

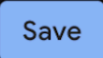
☐ HTTP endpoint URL

☒ Apps Script
Note: It's recommended to use a versioned deployment for Chat apps in staging or production environments, not a head deployment

☐ Cloud Pub/Sub

☐ Dialogflow

Deployment ID *

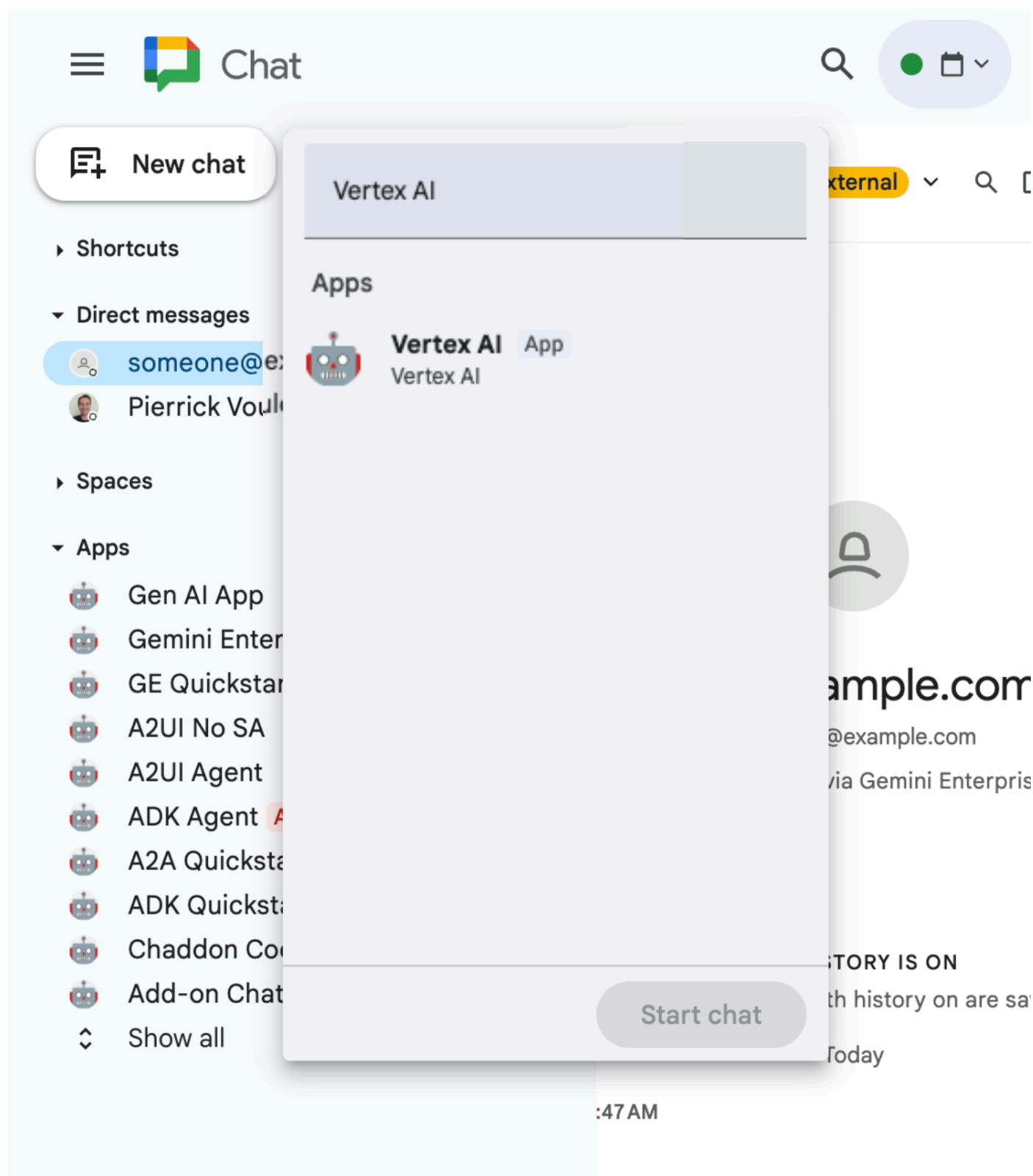
  

試用外掛程式

與即時外掛程式互動，確認外掛程式可以擷取資料，並根據情境回答問題。

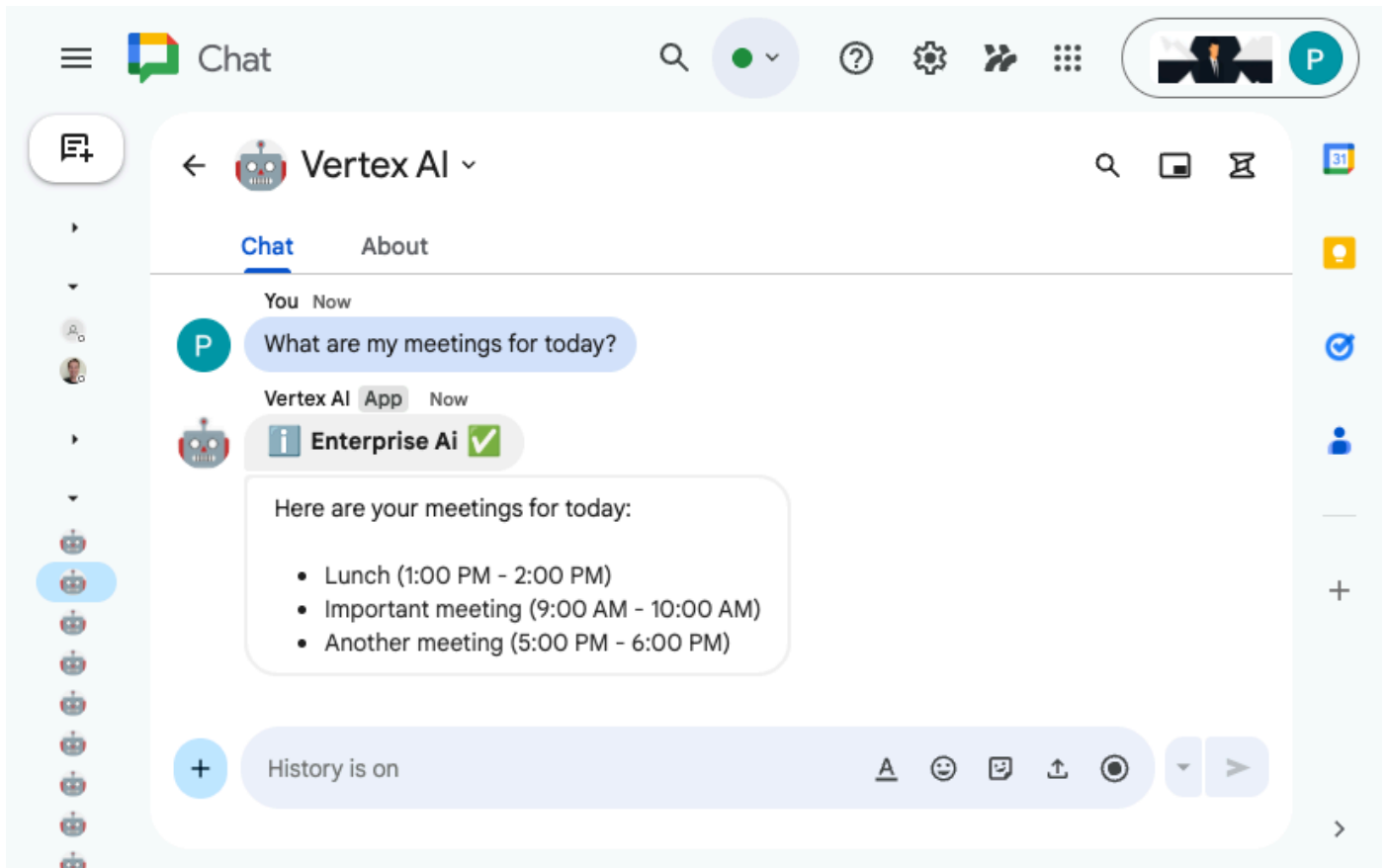
在新分頁中開啟 [Google Chat](#)，然後按照下列步驟操作：

1. 開啟與 Chat 應用程式 **Vertex AI** 互傳的即時訊息。



2. 按一下「設定」，然後完成驗證流程。

3. 輸入 `What are my meetings for today?`，然後按下 `enter` 鍵。**Vertex AI** Chat 應用程式應會回覆結果。



在新分頁中開啟 [Gmail](#)，然後按照下列步驟操作：

1. 傳送電子郵件給自己，並將「主旨」設為 `We need to talk`，內文設為 `Are you available today between 8 and 9 AM?`
2. 開啟新收到的電子郵件訊息。
3. 開啟 **Vertex AI** 外掛程式側欄。
4. 將「Message」（訊息）設為 `Do I have any meeting conflicts?`
5. 按一下 [傳送訊息]。
6. 答案會顯示在按鈕下方。

A screenshot of the Gmail web interface. The top navigation bar includes the Gmail logo, a search bar, and various settings and utility icons. On the left, a sidebar shows links to Mail, Chat, and Meet. The main content area displays an email from 'Pierrick' with the subject 'We need to talk'. Below the email text, there are buttons for 'Reply', 'Forward', and 'Share in chat'. A 'Vertex AI' chat window is overlaid on the right side of the screen, showing a message input field, a 'Send message' button, and a response from the AI stating that no meeting conflicts were found for the specified time.

6. 清除所用資源

刪除 Google Cloud 專案

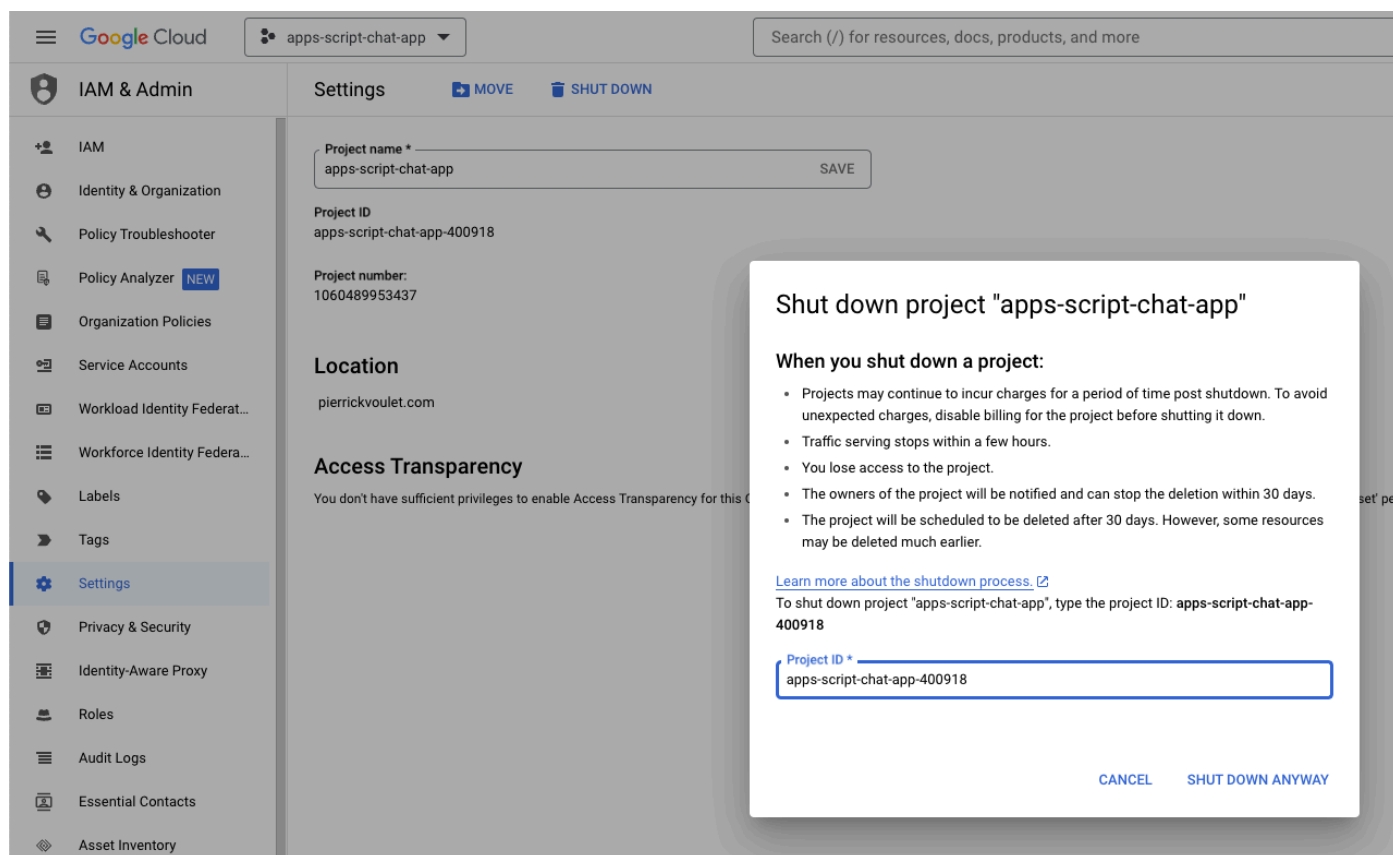
如要避免系統向您的 Google Cloud 帳戶收取本程式碼研究室所用資源的費用，建議您刪除 Google Cloud 專案。

注意：刪除 Google Cloud 專案會導致下列情況：

- 專案中的所有內容都會遭到刪除。如果您使用現有專案進行本程式碼研究室，刪除專案時也會一併刪除您在該專案中執行的其他任何工作。
- 自訂專案 ID 不復存在。您建立這個專案時，可能已建立日後使用的自訂專案 ID。如要保留使用該專案 ID 的網址 (如 appspot.com 上的網址)，請從專案刪除選定的資源，不要刪除整個專案。

在 [Google Cloud 控制台](#) 中，按照下列步驟操作：

- 依序點選「選單」圖示 ≡ > 「IAM 與管理」 > 「設定」。
- 按一下「Shut down」(關閉)。
- 輸入專案 ID。
- 按一下「仍要關閉」。



The screenshot shows the Google Cloud console interface. On the left is the 'IAM & Admin' sidebar with 'Settings' selected. The main panel displays the 'Settings' page for the project 'apps-script-chat-app'. A modal dialog box titled 'Shut down project "apps-script-chat-app"' is open in the foreground. The dialog contains a list of consequences when shutting down a project, a link to learn more about the shutdown process, and a text input field for the project ID. The project ID 'apps-script-chat-app-400918' is entered in the field. At the bottom of the dialog are 'CANCEL' and 'SHUT DOWN ANYWAY' buttons.

Shut down project "apps-script-chat-app"

When you shut down a project:

- Projects may continue to incur charges for a period of time post shutdown. To avoid unexpected charges, disable billing for the project before shutting it down.
- Traffic serving stops within a few hours.
- You lose access to the project.
- The owners of the project will be notified and can stop the deletion within 30 days.
- The project will be scheduled to be deleted after 30 days. However, some resources may be deleted much earlier.

[Learn more about the shutdown process.](#)

To shut down project "apps-script-chat-app", type the project ID: **apps-script-chat-app-400918**

Project ID *
apps-script-chat-app-400918

[CANCEL](#) [SHUT DOWN ANYWAY](#)

步驟 7 · 約 1 分鐘

7. 恭喜

恭喜！您建構的解決方案可讓 Vertex AI 和 Google Workspace 更緊密整合，為工作者提供強大助力！

後續步驟

本程式碼研究室只展示最典型的用途，但您可能想在解決方案中考慮許多擴充領域，例如：

- 使用 Gemini CLI 和 Antigravity 等 AI 輔助開發人員工具。
- 與其他代理架構和工具整合，例如自訂 MCP、自訂函式呼叫和生成式 UI。
- 與其他 AI 模型整合，包括託管於 Vertex AI 等專屬平台的自訂模型。
- 與其他代理程式整合，這些代理程式可託管在 Dialogflow 等專用平台，或由第三方透過 Cloud Marketplace 提供。
- 在 Cloud Marketplace 發布代理程式，協助團隊、機構或公開使用者。

瞭解詳情

開發人員可參考許多資源，例如 YouTube 影片、說明文件網站、程式碼範例和教學課程：

- [Google Cloud 開發人員中心](#)
 - [支援的產品 | Google Cloud MCP 伺服器](#)
 - [A2UI](#)
 - [Vertex AI Model Garden | Google Cloud](#)
 - [Vertex AI Agent Engine 簡介](#)
 - [取得答案和後續追蹤 | Vertex AI Search | Google Cloud 說明文件](#)
 - [透過 Google Cloud Marketplace 提供 AI 虛擬服務專員](#)
 - [Google Workspace 開發人員 YouTube 頻道 - 歡迎開發人員！](#)
 - [Google Workspace 開發人員網站](#)
 - [所有 Google Workspace 外掛程式範例的 GitHub 存放區](#)
-